

The *intexgral* package

v4.1.1

Valentin Dao*

August 1, 2026

Abstract

While integral composition is prevalent in $\LaTeX$, it often proves to be impractical. When the expression becomes complex, it is then difficult to modify its various elements in an illegible source code. To address this problem, the *intexgral* package offers a central macro whose only argument is the integrand. Everything else (symbol, limits, variables...) can be modified using a `\key=value` interface. Since this package is written in *expl3*, it depends on *l3kernel*.

*E-mail: vdao.texdev@gmail.com

Contents

Abstract	1
I Documentation	4
1 Package options	5
2 Main macro overview	5
3 List of keys	6
3.1 Integration limits	6
3.1.1 Defining the limits	6
3.1.2 Modify the style	7
3.2 Display modes	7
3.3 Symbol (and more about limits)	8
3.3.1 Selecting a symbol	8
3.3.2 Generating many integrals	8
3.3.3 Defining boundaries on an ad hoc basis	9
3.3.4 Recurring limits motif	9
3.4 Differentials	10
3.4.1 Specifying the integration variables	10
3.4.2 Including the Jacobian	10
3.4.3 Modifying the differential symbol	10
3.4.4 Creating a path integral	11
3.4.5 Adding an exponent	11
4 Additional macros	11
4.1 Custom differentials placement	11
4.1.1 Swapping the position	11
4.1.2 Choosing a custom position	12
4.2 Looking back at the <i>limits</i> key	13
4.2.1 Defining a keyword	13
4.2.2 Writing symmetric limits	14
4.3 Looking back at the <i>variables</i> key	15
4.3.1 Defining a keyword	15
4.4 Looking back at the <i>symbol</i> key	16
5 Special syntax	17
5.1 Presentation of the syntax	17
5.2 Functioning of the syntax	18
6 Additional parameters	19
Changelog	21
II Implementation	22
1 Initialisation	23
1.1 Package declaration	23

1.2	Preliminary verifications	23
2	Package options	24
2.1	Variable definition	24
2.2	Package option declaration	24
3	Messages	25
4	Useful variants	27
5	Internal variables	27
5.1	Tokens	27
5.2	Booleans	28
5.3	Sequences and clists	28
5.4	Property lists	29
5.5	Muskip	29
5.6	String	29
6	Auxiliary macros and sequences	30
6.1	Main sequences	30
6.2	Supplementary sequences	31
7	Integration limits	32
8	Variables	35
9	Special syntax	36
10	Typesetting the integral	38
10.1	Mode <i>default</i>	39
10.2	Mode <i>nested</i>	40
10.3	Mode <i>product</i>	41
10.4	Final typesetting	42
11	Key declaration	42
12	User-level macros	46
12.1	Keywords for limits	46
12.2	Keywords for variables	47
12.3	Keywords for symbols	48
12.4	Configuration macros	49
13	Package configuration	50
13.1	Symbol	50
13.2	Limits	50
13.3	Variables	51
13.4	Default parameters	51
	Index	52

License

© 2025 Valentin Dao, published under the ~~La~~TeX Project Public License (LPPL) 1.3c

Fonts

Chronicle Text G3 Roman

© 2002, 2007 Hoefler & Frere-Jones.

Whitney-Medium

© 1996, 2009 Hoefler & Frere-Jones.

Monospace Argon Medium

© 2023, GitHub

Repository

 [See GitHub repository.](#)

Acknowledgements

Many thanks to Plante and Slurpy for accompanying me on this adventure of learning T_EX, your pieces of advice were priceless. I'd also like to thank Anthony, who has always kindly reviewed the new versions and given me ideas for those to come.

PART I

DOCUMENTATION

1 Package options

`\intexgralsetup` `\intexgralsetup` {<package options>}

new: v2.0.0

These options can be declared in a classic way with `\usepackage` or with `\intexgralsetup` in the preamble.

`invert-limits` `invert-limits`=<`true` | `false`>

This key inverts the limit order convention. The entirety of the document can only follow one convention.

`invert-diff` `invert-diff`=<`true` | `false`>

update: v3.0.0

In physical sciences, it is common to see the differentials placed before the integrand. This key hence swaps their position.

`limits-mode` `limits-mode`=<`limits` | `nolimits`>

new: v3.0.0

Applies `\limits` or `\nolimits` to all integrals.

`italic` `italic`=<`true` | `false`>

`upright` `upright`=<`true` | `false`>

update: v4.0.0

Applies an upright or italic “d” for the differentials.

Remark: Since the version v4.0.0, the *derivative* package is no longer used to manage differentials. The number of options is therefore more limited but should suit most use cases in the context of integrals.

2 Main macro overview

`\integral` `\integral` [<list of keys>] {<integrand>}

update: v3.0.0

This macro is used to typeset integrals. It must, of course, be used in math mode only. Here is a first example in its simplest form:

```
\begin{equation}
\integral{x}
\end{equation}
```

$$\int x \, dx \tag{1}$$

Since this macro focuses on the use of keys, the first part of this documentation will present them, grouping them by domain. The second part will introduce a few

additional macros that complement the use of certain keys. The rest of this document will outline the other features of the package.

3 List of keys

3.1 Integration limits

3.1.1 Defining the limits

`limits` `limits=` {<mixed-list>}, <keyword>
`limits*` `limits*=` {<mixed-list>}, <keyword>

This key determines the integration limits to be used. In its simplest form, the value of the key takes as its argument a list of elements separated by commas (*comma-separated values* or <csv-list>). This simplification of the input requires a convention: the first and second elements of the list will correspond to the lower and upper limits, respectively. This convention can be reversed using `invert-limits` as seen in section 1.

```
\begin{equation}
  \integral[limits={1, 10}]{f(x)}
\end{equation}
```

$$\int_1^{10} f(x) \, dx \quad (2)$$

The notable advantage of the `limits` key is that you can specify the integration limits for several integrals, and the package automatically typesets the corresponding symbols. To separate pairs of limits, we use a semicolon (*semicolon-separated values* or <ssv-list>).

```
\begin{equation}
  \integral[limits={1, 2; 3, 4}, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\int_1^2 \int_3^4 f(x, y) \, dx \, dy \quad (3)$$

It is also possible to specify predefined limits using keywords (see ??). Finally, the starred variant allows the limits of the integral to be expressed as an interval. The direction of the brackets automatically adjusts itself to the presence of $\pm\infty$.

```
\begin{equation}
  \integral[limits*={1, 10}]{f(x)}
\end{equation}
```

$$\int_{[1,10]} f(x) \, dx \quad (4)$$

^oWe therefore mean by <mixed-list> that `limits` can accept a set of <csv-list> in a more general <ssv-list>.

3.1.2 Modify the style

`limits-mode` `limits-mode=<limits | nolimits>`

new: v4.1.0 Homonymous key to the one seen in section 1, which allows you to modify the style locally rather than globally.

3.2 Display modes

`mode` `mode=<default | nested | product>`

new: v3.0.0 *Solely* when the `limits` key is used, it is possible to choose between three distinct display styles. Each of these is fully compatible with the `invert-diff` option.

default Typesets all the symbols, then the integrand, then the variables. As the macro operates by default in this mode, it is not necessary to use the key with this value.

nested Typesets a nested integral where the symbol and integrand alternate before all the variables are written.

product Typesets a product of integrals where the symbol, integrand, and variables alternate.

Of course, nothing prevents you from using the `\integral` macro several times in a row to produce the same result as in the example below. However, for those who would prefer to obtain the result with a single macro, this method is permitted.

Important: for the last two modes, the integral will be *cut off* in a certain way. In order to perform this action correctly, the same method must be used as with `limits`, i.e. by using a semicolon.

MODE **default**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={x, y, z}, mode=default]{xyz}
\end{equation}
```

$$\int_1^2 \int_3^4 \int_5^6 xyz \, dx \, dy \, dz \quad (5)$$

MODE **nested**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={z, y, x}, mode=nested]{x;y;z}
\end{equation}
```

$$\int_1^2 x \int_3^4 y \int_5^6 z \, dz \, dy \, dx \quad (6)$$

MODE **product**

```
\begin{equation}
  \integral[limits={1, 2; 3, 4; 5, 6}, variables={x, y, z}, mode=product]{x;y;z}
\end{equation}
```

$$\int_1^2 x \, dx \int_3^4 y \, dy \int_5^6 z \, dz \quad (7)$$

To function correctly, it is clear that the keys `limits` and `variables` (see 3.4.1), as well as the integrand, must have the same number of elements.

3.3 Symbol (and more about limits)

The `limits(*)` keys are very useful when typesetting a definite integral. However, this only covers final expressions where the limits of each variable have been specified. For more general cases — indefinite integrals — they are relatively inconvenient. To typeset a double integral over a surface S , one would have to write `limits={,;S,}`. Needless to say, this is rather inconvenient for the user and not optimal for the package. Furthermore, the glyph would not be correct. The set of keys below therefore offers an easy way to modify both the symbol and the bounds.

3.3.1 Selecting a symbol

symbol

`symbol=<control sequence>`

update: v3.0.0

This key accepts a macro designating an integral symbol. Any symbol previously defined by a control sequence is acceptable. If you attempt to use one that is not defined, it will be substituted with `\iint` and a warning message will be issued.

Remark: aside from checking whether the macro exists, no specific checks are performed, and the value of the key is used as-is to typeset the symbol. This implies that the symbol will depend on how it is defined. For example, *amsmath* and *unicode-math* do not define `\iint` in the same way. The result will therefore naturally be different: one combines two `\iint`s with a certain spacing to define `\iint`, while the other defines the actual glyph U+222C.

```
\begin{equation}
  \integral[symbol=\iint, llimit=S, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\iint_S f(x, y) \, dx \, dy \quad (8)$$

3.3.2 Generating many integrals

nint `nint=<integer>`

This key accepts an integer n which allows n integrals to be composed. It is advisable to use this key only if the number of symbols exceeds 4 to favour the glyphs defined by the mathematical font used.


```
\begin{equation}
\int[nint=5, llimit=\Omega, variables={x_1, x_2, x_3, x_4, x_5}]{f(x_1, x_2,
x_3, x_4, x_5)}
\end{equation}
```

$$\iiint\limits_{\Omega} f(x_1, x_2, x_3, x_4, x_5) dx_1 dx_2 dx_3 dx_4 dx_5 \quad (9)$$

3.3.3 Defining boundaries on an ad hoc basis

```
llimit    llimit=<lower limit>
ulimit    ulimit=<upper limit>
```

update: v3.0.0 These two keys allow you to specify the lower and upper limits, respectively. They are only suitable if a single symbol is displayed. If you need to specify both limits, the `limits` key should be used instead.

```
\begin{equation}
\int[llimit={x^2 + y^2 \leq 1}, variables={x, y}]{f(x, y)}
\end{equation}
```

$$\int_{x^2+y^2 \leq 1} f(x, y) dx dy \quad (10)$$

3.3.4 Recurring limits motif

Limits sometimes follow common patterns that can be generalised with keys, thus avoiding overloading the argument of `llimit`.

```
domain    domain= {<*-list>}
domain*    domain*= {<*-list>}
```

update: v3.0.0 These keys accept a list delimited by an asterisk. Each element of the list is then parsed as follows:

- The first token of the item is passed `\mathbb` (preceded by `\uppercase` in case the SHIFT key is too far away for your fingers).
- The remaining tokens — which may be empty — are placed as superscript (or subscript for the starred variant of the key).

```
\begin{equation}
\int[symbol=\iint, domain={\mathbb R*\mathbb R}, variables={x, y}]{xy}
\end{equation}
```

$$\iint_{\mathbb R \times \mathbb R} xy dx dy \quad (11)$$

```
boundary    boundary= {<lower limit>}
```

This key simply places the symbol ∂ before the lower limit.

```
\begin{equation}
\int_{\partial S} G(\vec{r}) \cdot d\vec{r}
\end{equation}
```

$$\oint_{\partial S} G(\vec{r}) \cdot d\vec{r} \quad (12)$$

3.4 Differentials

3.4.1 Specifying the integration variables

variables `variables={⟨csv-list⟩}, ⟨keyword⟩, none`

update: v3.0.0 This key allows you to define the variables of the integral in the form of a `⟨csv-list⟩`. Like `limits`, the key can accept a keyword as an argument. This behaviour is also explained later (see ??).

Remark: if no variable is specified, i.e. `variables` is not called, the package automatically sets “ dx ” (or “ $d\vec{r}$ ” if `diff-vec` is active). Furthermore, if `none` is passed as a value, no variable will be displayed.

```
\begin{equation}
\int_{\partial S} t^2 dt
\end{equation}
```

$$\int t^2 dt \quad (13)$$

3.4.2 Including the Jacobian

jacobian `jacobian`

This key enables the display of the Jacobian when defined by `\NewVariableKeyword`.

```
\begin{equation}
\int_0^R \int_0^{2\pi} \int_0^\pi r^2 \sin \theta dr d\theta d\phi
\end{equation}
```

$$\int_0^R r^2 dr \int_0^{2\pi} \sin \theta d\theta \int_0^\pi d\phi \quad (14)$$

3.4.3 Modifying the differential symbol

diff-symb `diff-symb=⟨control sequence⟩`

This key allows you to modify the symbol or style of the differentials.

```
\begin{equation}
\int_{\partial S} q(t) dt
\end{equation}
```

$$\int e^{+\frac{iS[q(t)]}{\hbar}} \mathcal{D}q(t) \quad (15)$$

3.4.4 Creating a path integral

diff-vec `diff-vec`

This key applies a vector to each integration variable along with a centred dot. It is only compatible with the `default` mode. Using it with the `nested` or `product` options will have no effect.

```
\begin{equation}
W = \int_C \vec{F} \cdot d\vec{s}
\end{equation}
```

$$W = \int_C \vec{F} \cdot d\vec{s} \quad (16)$$

3.4.5 Adding an exponent

diff-order `diff-order=<csv-list>`

This key places an exponent on each differential.

```
\begin{equation}
S[\phi] = \int \mathcal{L}(\phi, \partial_\mu \phi, x^\mu) d^4x
\end{equation}
```

$$S[\phi] = \int \mathcal{L}(\phi, \partial_\mu \phi, x^\mu) d^4x \quad (17)$$

4 Additional macros

In addition to the wide range of keys defined by the package, a few macros also enhance their uses.

4.1 Custom differentials placement

4.1.1 Swapping the position

\invertdiff `\invertdiff`

new: v4.1.0

This macro inverts the status of the `invert-diff` option, whether it is initialised to `true` or `false`.

```

\invertdiff
\begin{equation}
\int f(x)
\end{equation}
\invertdiff
\begin{equation}
\int f(\omega)
\end{equation}

```

$$\int dx f(x) \quad (18)$$

$$\int f(\omega) d\omega \quad (19)$$

4.1.2 Choosing a custom position

\differentials \differentials

Although the `invert-diff` option exists, it is often desirable to be able to place differentials wherever one wishes. For example, they are frequently seen in the numerator of a fraction when the numerator is 1. To meet this need, the package provides the macro `\differentials`, whose functioning varies slightly from one display mode to another:

- **default**: typesets all the differentials at once. It is compatible with `invert-diff/\invertdiff`.
- **nested**: typesets all the differentials at once. The macro is *not* defined if `invert-diff/\invertdiff` is active.
- **product**: typesets a single differential at a time. It will therefore need to be repeated between each semicolon. It is also compatible with `invert-diff/\invertdiff`.

```

\begin{gather}
\int \frac{\differentials}{x} \\
\int \frac{\differentials}{(2\pi)^4} \frac{e^{-ip \cdot (x-y)}}{p^2 - m^2 + i\epsilon}
\end{gather}

```

$$\int \frac{dx}{x} \quad (20)$$

$$\langle 0 | T\{\phi(x)\phi(y)\} | 0 \rangle = \int \frac{d^4p}{(2\pi)^4} \frac{e^{-ip \cdot (x-y)}}{p^2 - m^2 + i\epsilon} \quad (21)$$

4.2 Looking back at the *limits* key

4.2.1 Defining a keyword

```
\NewLimitsKeyword \NewLimitsKeyword {\keyword} {\limits}  
\RenewLimitsKeyword  
\ProvideLimitsKeyword  
\DeclareLimitsKeyword
```

update: v4.0.0

It is common to have to specify the same limits in an integral. To make them easier to write, the `limits(*)` keys accept, in addition to explicit limits, a keyword designating a pair of them predefined by one of these four macros:

- `\NewLimitsKeyword` Creates a new keyword and issues an error if it already exists.
- `\RenewLimitsKeyword` Redefines a keyword and issues an error if it does not already exist.
- `\ProvideLimitsKeyword` Only creates a new keyword if it does not already exist. No message will be issued if this is the case.
- `\DeclareLimitsKeyword` Creates a new keyword regardless, overwriting any previous definition.

When entering a new keyword, you must follow the convention for the order of delimiters. For example, you can write:

```
\usepackage{intexgral}  
\NewLimitsKeyword{key1}{A, B}  
\intexgralsetup{invert-limits=true}  
\NewLimitsKeyword{key2}{B, A}
```

`key1` and `key2` will then produce the same integral. Here is the list of keywords already defined by the package and the limits they contain:

<code>ab</code>	a, b
<code>unit</code>	$0, 1$
<code>real</code>	$-\infty, +\infty$
<code>positive</code>	$0, +\infty$
<code>negative</code>	$-\infty, 0$
<code>circle</code>	$0, 2\pi$
<code>scircle</code>	$0, \pi$
<code>qcircle</code>	$0, \pi/2$
<code>height</code>	$0, H$
<code>radius</code>	$0, R$
<code>length</code>	$0, L$
<code>time</code>	$0, T$

Remark: keywords contain both limits of an integral at the same time. They must therefore be preceded and/or followed by semicolons.

We could therefore modify the expression of an integral as follows:

```
\begin{equation}  
  \integral[limits={height;circle;radius}, variables={\rho, \theta, z}]{\rho}  
\end{equation}
```

$$\int_0^H \int_0^{2\pi} \int_0^R \rho \, d\rho \, d\theta \, dz \quad (22)$$

It is entirely possible to combine these keywords with the more explicit syntax of `limits`.

```
\begin{equation}
  \integral[limits={height;0,W,length}, variables={x, y, z}]{\kappa(T)\nabla T \cdot
  \mathbf{n}}
\end{equation}
```

$$\int_0^H \int_0^W \int_0^L \kappa(T) \nabla T \cdot \mathbf{n} \, dx \, dy \, dz \quad (23)$$

4.2.2 Writing symmetric limits

`limits` `limits= {<+-limit>}`

new: v4.1.0 When symmetric limits need to be specified, the package can identify them using a syntax specific to the `limits` key. To do this, simply insert the tokens `+-` before the relevant limit.

Remark: the same remark as above applies. Moreover, this syntax *is not* compatible with the keywords seen just before. The goal is to allow the input of occasional symmetric bounds that do not justify the creation of a keyword for a single use.

```
\begin{equation}
  a_n = \frac{1}{\pi} \integral[limits={+-\pi}]{f(x) \cos(x)}
\end{equation}
```

$$a_n = \frac{1}{\pi} \int_{-\pi}^{+\pi} f(x) \cos(x) \, dx \quad (24)$$

4.3 Looking back at the *variables* key

4.3.1 Defining a keyword

```

\NewVariableKeyword \NewVariableKeyword {⟨keyword⟩} {⟨variables⟩} [⟨jacobian⟩]
\RenewVariableKeyword
\ProvideVariableKeyword
\DeclareVariableKeyword

```

update: v4.0.0

Similarly, the same groups of variables often reappear. We can therefore also define keywords containing a set of pre-registered variables. The variants of this macro behave in the same way as for `\NewLimitsKeyword`. The only difference is that here it is possible to define a Jacobian, in the form of a `⟨csv-list⟩` if necessary. Its display is then controlled by the `jacobian` key as explained above. Here is the list of variable keywords already defined by the package, with the Jacobian for some of them:

```

cartesian     $x, y, z$ 
planar        $x, y$ 
polar         $r, \theta (r)$ 
cylindrical   $r, \theta, z (r)$ 
cylindrical*  $\rho, \theta, z (\rho)$ 
spherical     $r, \theta, \phi (r^2, \sin \theta)$ 

```

```

\begin{equation}
  \int\limits_V f(r, \theta, \phi) \, dr \, d\theta \, d\phi
\end{equation}

```

$$\iiint_V f(r, \theta, \phi) \, dr \, d\theta \, d\phi \quad (25)$$

4.4 Looking back at the *symbol* key

```

\NewSymbolKeyword \NewSymbolKeyword {\key} {\symbol}
\RenewSymbolKeyword
\ProvideSymbolKeyword
\DeclareSymbolKeyword

```

new: v3.0.0

In order to typeset indefinite integrals, the package essentially offers two keys: `symbol` and `llimit`. Although they are simple to use, it is still laborious to write both of them with their values. This is why *intexgral* provides so-called *shortcut* keys, whose purpose is to combine the selection of the symbol and the limits. Unlike the two previous macros, this involves creating entirely new keys, rather than simply specific values assigned to `symbol`. On their own, these keys modify the integral symbol. The value of these keys corresponds to the lower bound. Here is the list of all *symbol-keys* defined by the package:

<code>single</code>	used symbol = <code>\int</code>
<code>double</code>	used symbol = <code>\iint</code>
<code>triple</code>	used symbol = <code>\iiint</code>
<code>quadruple</code>	used symbol = <code>\iiiiint</code>
<code>contour</code>	used symbol = <code>\oint</code>
<code>surface</code>	used symbol = <code>\oiint</code>
<code>volume</code>	used symbol = <code>\oiiint</code>

Important: allowing users to create their own keys for the `\integral` macro poses a risk that will be explained in more detail in the next section. For now, just remember that all these macros¹ will issue a warning message if you attempt to create a new *symbol-key* with the same name as a group of defined limits.

```

\begin{gather}
\integral[double=S, variables=planar]{f(x, y)}{\[1cm]}
\integral[contour=\mathcal{C}, diff-vec]{f(\vec{r})}
\end{gather}

```

$$\iint_S f(x, y) \, dx \, dy \quad (26)$$

$$\oint_C f(\vec{r}) \cdot d\vec{r} \quad (27)$$

5 Special syntax

5.1 Presentation of the syntax

`\integral` `\integral` [`\limits:variables(+j):mode`] {`\integrand`}

new: v3.0.0

With regard to definite integrals, the user will be required to use a maximum of four keys to compose them: `limits`, `variables`, `mode` and `jacobian`. However, they can be criticised for the same reason as before; writing these four keys, which can take quite long arguments, runs counter to the main objective of this package. Thus, the user can designate as an optional argument of `\integral` a syntax called *special* that is unique to `intexgral`. The logic is as follows:

- You specify a *valid* argument for the `limits` key
- You specify a *valid* argument for the `variables` key
- You specify a *valid* argument for the `mode` key

And you separate everything with a colon. Let's look at an example to better understand.

```
\begin{equation}
  \integral[1, 2; 4, 5:y, x:nested]{x; y}
\end{equation}
```

$$\int_1^2 x \int_4^5 y \, dy \, dx \quad (28)$$

We mean by *valid argument* that all forms of values accepted by the four keys can also be likewise adopted by the special syntax. This hence includes keywords.

```
\begin{equation}
  \integral[radius;circle;scircle:spherical:product]{r;\theta;\phi}
\end{equation}
```

$$\int_0^R r \, dr \int_0^{2\pi} \theta \, d\theta \int_0^\pi \phi \, d\phi \quad (29)$$

To include the Jacobian, simply write `+j` after the argument of `variables`. The mode can also be indicated using only an initial.

```
\begin{equation}
  \integral[radius;circle;scircle:spherical+j:p]{;};
\end{equation}
```

$$\int_0^R r^2 \, dr \int_0^{2\pi} \sin \theta \, d\theta \int_0^\pi d\phi \quad (30)$$

It is important to reiterate that in the classic configuration of the optional argument, it is of course not mandatory to specify the four keys mentioned. The same applies to this syntax. Since the key `mode` is not necessary for `default`, this part can be omitted.

```
\begin{equation}
\integral[1, 2; 3, 4:x, y]{xy}
\end{equation}
```

$$\int_1^2 \int_3^4 xy \, dx \, dy \quad (31)$$

You can also take advantage of the automatic placement of “dx” with this syntax, ignoring `variables`.

```
\begin{equation}
\integral[1, 10]{x}
\end{equation}
```

$$\int_1^{10} x \, dx \quad (32)$$

One notable special case of this syntax is that if you want to modify the variable without specifying the bounds, you can do so more quickly as follows:

```
\begin{equation}
\integral[:z]{f(z)}
\end{equation}
```

$$\int f(z) \, dz \quad (33)$$

Even if it is not recommended.

5.2 Functioning of the syntax

From the user’s perspective, it is relatively simple to choose which syntax to adopt. However, the package must be able to distinguish between the two. Without going too deeply into the details² of the implementation, it is worth noting that the package attempts to extract the first key from the optional argument³ and checks if it exists. This is why, if you create a key with `\NewSymbolKeyword` whose name may probably serve as integration bounds, `intexgral` may falsely evaluate the nature of the optional argument. In this kind of situation, the interpretation of the keys will be incorrect and the result will be wrong.

²The more courageous among you are still invited to read the source code.

³Or at least, the group of tokens it thinks correspond to a key.

6 Additional parameters

`\IntegralSetup` `\IntegralSetup {⟨list of parameters⟩}`

new: v3.0.0
update: v4.0.0

This macro allows, under the syntax `⟨key=value⟩`, to modify parameters related to certain keys. All assignments made are local and the macro can therefore be placed in a group if needed. Here is an example of the macro with the default assignments of the package:

```
\IntegralSetup{
  diff-symb    = \l_@@_default_differential_symbol_tl, % see at the beginning of the
                implementation
  defaultvar   = {x},
  defaultvar*  = {r},
  varsep       = \thinmuskip,
  diffsep      = \thinmuskip,
  innersymbsep = {-5mu},
  postsymbsep  = {-1mu},
  vectorstyle  = \vec,
  domainstyle  = \mathbb,
  novar        = false
}
```

`diff-symb` `diff-symb=⟨control sequence⟩`

new: v4.0.0

This key controls the symbol used for differentials. If the option `italic` is used, `\math-normal` is applied (see at the beginning of the implementation). Unlike the homonymous key of `\integral`, this one acts at a more global level.

`defaultvar` `defaultvar={⟨variables⟩}`

`defaultvar*`

new: v3.0.0

This key modifies the variable automatically inserted when `variables` is not used. The argument may correspond to a keyword defined by `\NewVariableKeyword`. The starred variant of the key controls the integration parameter to be typeset with `diff-vec`.

`varsep` `varsep=⟨muexpr⟩`

new: v4.0.0

This key modifies the spacing between the integrand (or the Jacobian if it is displayed) and the variables.

`diffsep` `diffsep=⟨muexpr⟩`

new: v4.0.0

This key modifies the spacing between the differentials themselves.

`innersymbsep` `innersymbsep=⟨muexpr⟩`

new: v4.0.0

When the `default` mode is in effect, that is to say when the `limits` key is used, the package inserts a gap between the symbols to slightly bring them together. This key therefore allows you to adjust this spacing.

`postsymbsep` `postsymbsep=⟨muexpr⟩`

new: v4.0.0

This key controls the spacing between the symbol and the element that follows (integrand or differential depending on `invert-limits`).

`vectorstyle` `vectorstyle=⟨control sequence⟩`

new: v3.0.0

This key modifies the macro to be applied to variables when `diff-vec` is in action. It is therefore possible to alter the style of the vector: bold, underlined, or even another style from a particular package, `esvect` for example.

domainstyle `domainstyle=<control sequence>`

new: v3.0.0 Similarly, it is possible to change the macro to be applied for the `domain(*)` keys.

novar `novar=<true | false>`

new: v3.0.0
update: v4.0.0 When a large part of the document requires not to include the differentials, it is possible to extend the duration of the more punctual action of `variables=none`.

Changelog

4.1.1 (01-08-2026)

Fixed

- ▶ Issue with `\differentials` used in `product` mode ([issue #14](#)).
- ▶ Issue with `\RenewSymbolKeyword` and `\DeclareSymbolKeyword` ([issue #15](#)).
- ▶ Issue with `\RenewVariableKeyword` and `\DeclareVariableKeyword` ([issue #17](#)).

4.1.0 (27-07-2026)

Added

- ▶ `limits-mode` key for `\integral`.
- ▶ `\invertdiff` macro.
- ▶ The `+-` syntax for limits.

Changed

- ▶ Source code optimisation.

4.0.0 (06-06-2026)

Added

- ▶ Detailed implementation.
- ▶ Keys `varsep`, `diffsep`, `postsymbsep` and `diff-order`.
- ▶ Variable keyword `cylindrical*`.

Changed

- ▶ The `derivative` is no longer a dependency. The differentials are now managed internally.
- ▶ Key `symbolskip`, renamed in `innersymbsep`.
- ▶ Key `hide-diff`, renamed in `novar`.

Removed

- ▶ Keys `diff-star` and `diff-options`.

3.0.1 (02-01-2026)

Fixed

- ▶ Bug with jacobian in special syntax ([issue #3](#)).
- ▶ French and English documentations ([issue #4](#), [issue #6](#) and [issue #7](#)).

Changed

- ▶ Limits keywords `positive` and `real` now contain a `+` sign ([issue #5](#)).

3.0.0 (24-12-2025)

Added

- ▶ Special syntax.
- ▶ Keys `domain*` and `mode`.

- ▶ Macros `\IntegralSetup` and `\NewSymbolKeyword`.

Removed

- ▶ Macros `\defaultdiff`, `\defaultdiff`, `\defaultvdiff` and `\vdiffstyle` in favour of `\IntegralSetup`.
- ▶ Keys controlling both symbol and limit, now managed at a higher level by `\NewSymbolKeyword`.
- ▶ `int-split` key, in favour of `mode`.
- ▶ `\NewIntegralSymbol` macro.

Changed

- ▶ The names of some keys (`variable` to `variables`, `lower-lim` and `upper-lim` to `llimit` and `ulimit`, `int-symb` to `symbol`, `invert-differentials` and `hide-differentials` to `invert-diff` and `hide-diff`).
- ▶ `jacobian` and `diff-star` no longer require a boolean value. Simply writing them will now enable their features.
- ▶ `hide-diff` assigned to a local option rather than a package one.
- ▶ `limits-mode` assigned to a package option rather than a macro key

v2.0.1 (13-09-2025)

Fixed

- ▶ Compatibility issue between `unicode-math` and `amssymb` depending on the loading order ([problem #2](#)).

2.0.0 (09-09-2025)

Added

- ▶ Macros `\intexgralsetup`, `\defaultdiff`, `\defaultvdiff` and `\vdiffstyle`.

Changed

- ▶ Warning messages related to non-existing symbols. They are now only triggered when the integral is typeset.

Removed

- ▶ `diff-vec-style` key in favour of `\vdiffstyle`.
- ▶ Warning message about misuse of semi-colon in conjunction with the `int-split` key.

Fixed

- ▶ Bug where the integrand was not reset when `\integral` was used successively in the same $\text{\TeX}$ group ([details here](#)).
- ▶ `expl3` log leftovers that shouldn't have been there.

1.1.0 (29-07-2025)

Added

- ▶ Starred variant for the keys controlling both symbol and limit (keys `single`, `contour` etc).

1.0.0 (26-07-2025) — Initial version.

PART II

IMPLEMENTATION

1 Initialisation

```
1 ⟨*package⟩
2 ⟨@@=intexgral⟩
```

1.1 Package declaration

```
3 \NeedsTeXFormat{LaTeX2e}
4 \RequirePackage{expl3}[2025-05-14]
5
6 \def\intexgral@module{intexgral}
7 \def\intexgral@version{v4.1.1}
8 \def\intexgral@date{2026-08-01}
9 \def\intexgral@description{A LaTeX package for typesetting integrals}
10
11 \ProvidesExplPackage
12   \intexgral@module
13   \intexgral@date
14   \intexgral@version
15   \intexgral@description
```

1.2 Preliminary verifications

We check that the version of `expl3` is recent enough before continuing. The package requires the macro `\tl_if_regex_match:nnTF`, available from December 2024.

```
16 \msg_new:nnn { intexgral } { expl3-too-old }
17 {
18   Your~expl3~installation~is~too~old,~
19   "intexgral"~requires~expl3~dated~2024-12-08~or~later.\iow_newline:
20   Package~loading~has~been~aborted.\iow_newline:
21   \msg_see_documentation_text:n { intexgral }
22 }
23
24 \@ifpackagelater{expl3}{2024-12-08}{}
25 { \msg_critical:nn { intexgral } { expl3-too-old } }
```

Since `intexgral` uses `\mathbb` for the keys `domain(*)`, we must ensure that the macro exists. The verification is placed in a *hook* executed at the beginning of the document in case `amsfonts` or any package defining `\mathbb` is loaded after `intexgral`. This serves particularly to avoid conflicts between packages (`unicode-math` and `amssymb` for example).

```
26 \hook_gput_code:nnn { begindocument/before } { . }
27 { \cs_if_exist:NF \mathbb { \RequirePackage{amsfonts} } }
```

2 Package options

2.1 Variable definition

`\l_@@_invert_limits_bool`

Boolean inverting the order of the limits.

28 `\bool_new:N \l__intexgral_invert_limits_bool`

`\l_@@_deactivate_variables_bool`

Determines whether the differentials should be displayed or not.

29 `\bool_new:N \l__intexgral_deactivate_variables_bool`

`\l_@@_invert_differential_bool`

Determines the order of display for the differentials.

30 `\bool_new:N \l__intexgral_invert_differential_bool`

`\l_@@_default_differential_symbol_tl`

Contains the default symbol for differentials.

31 `\tl_new:N \l__intexgral_default_differential_symbol_tl`

`\l_@@_limits_style_tl`

Contains the $\TeX$ primitive to use for the limits style.

32 `\tl_new:N \l__intexgral_limits_style_tl`

2.2 Package option declaration

We can now define the package options before loading them.

```
33 \keys_define:nn { intexgral }
34   {
35     invert-limits .bool_set:N = \l__intexgral_invert_limits_bool,
36     invert-limits .usage:n    = { preamble },
37     invert-limits .initial:n  = { false },
38
39     invert-diff .bool_set:N = \l__intexgral_invert_differential_bool,
40     invert-diff .usage:n    = { preamble },
41     invert-diff .initial:n  = { false },
42
43     limits-mode .choice:,
44     limits-mode / limits .code:n =
45       { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_limits:D },
46     limits-mode / nolimits .code:n =
47       { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_nolimits:D },
48
49     limits-mode .usage:n    = { preamble },
50     limits-mode .initial:n  = { nolimits },
51
52     italic .choice:,
53     italic / true .code:n =
54       {
55         \tl_set:Nn \l__intexgral_default_differential_symbol_tl
```



```

56         { \mathnormal{d} }
57     },
58     italic / false .code:n =
59     {
60         \tl_set:Nn \l__intexgral_default_differential_symbol_tl
61         { \mathrm{d} }
62     },
63     italic .usage:n = { preamble },
64     italic .initial:n = { false },
65
66     upright .choice:,
67     upright / true .code:n =
68     {
69         \tl_set:Nn \l__intexgral_default_differential_symbol_tl
70         { \mathrm{d} }
71     },
72     upright / false .code:n =
73     {
74         \tl_set:Nn \l__intexgral_default_differential_symbol_tl
75         { \mathnormal{d} }
76     },
77     upright .usage:n = { preamble },
78     upright .initial:n = { true }
79 }
80
81 \ProcessKeyOptions[intexgral]

```

3 Messages

The next part of the implementation defines all the warning and error messages.

`\g_@@_integral_number_int`

We start by creating a global counter to number the integrals in the document.

```
82 \int_gzero_new:N \g__intexgral_integral_number_int
```

`\@@_warning_msg_header: ★`

We can now define a macro that acts as a header to warning messages. It uses the counter defined just before.

```

83 \cs_new:Nn \__intexgral_warning_msg_header: {
84     (
85         integral~no.~
86         \int_use:N\g__intexgral_integral_number_int\c_space_tl
87         \msg_line_context:
88     )
89     \iow_newline:
90 }

```

All the messages are then defined.

```

91 \msg_new:nnnn { intexgral } { symb-key-alr-def }
92 { Symbol~key~"\tl_trim_spaces:n{#1}"~is~already~defined. }
93 { Use~\token_to_str:N \RenewSymbolKeyword\ instead. }
94
95 \msg_new:nnnn { intexgral } { symb-key-not-def }
96 { Symbol~key~"\tl_trim_spaces:n{#1}"~is~not~defined. }
97 { Use~\token_to_str:N \NewSymbolKeyword\ instead. }

```

```

98
99 \msg_new:nnnn { intexgral } { diff-group-alr-def }
100 { Differential~group~"\tl_trim_spaces:n{#1}"~is~already~defined. }
101 { Use~\token_to_str:N \RenewVarKeyword\ instead. }
102
103 \msg_new:nnnn { intexgral } { diff-group-not-def }
104 { Differential~group~"\tl_trim_spaces:n{#1}"~is~not~defined. }
105 { Use~\token_to_str:N \NewVarKeyword\ instead. }
106
107 \msg_new:nnnn { intexgral } { lim-group-alr-def }
108 { Limits~group~"\tl_trim_spaces:n{#1}"~is~already~defined. }
109 { Use~\token_to_str:N \RenewLimitsKeyword\ instead. }
110
111 \msg_new:nnnn { intexgral } { lim-group-not-def }
112 { Limits~group~"\tl_trim_spaces:n{#1}"~is~not~defined. }
113 { Use~\NewLimitsKeyword\ instead. }
114
115 \msg_new:nnn { intexgral } { key-exists-for-lim }
116 {
117     Symbol~key~already~defined~with~
118     \token_to_str:N \NewLimitsKeyword\ \__intexgral_warning_msg_header:
119     Key~"#1"~already~exists~for~predefined~limits.~
120     This~can~disrupt~the~functioning~of~the~special~syntax.
121 }
122
123 \msg_new:nnn { intexgral } { unknown-symb }
124 {
125     Unknown~integral~symbol~\__intexgral_warning_msg_header:
126     The~symbol~"\tl_trim_spaces:n{#1}"~has~been~replaced~with~
127     \token_to_str:N\int.
128 }
129
130 \msg_new:nnn { intexgral } { no-jacobian }
131 {
132     Jacobian~unavailable~\__intexgral_warning_msg_header:
133     A~Jacobian~was~requested~for~the~"#1"~keyword,
134     ~but~none~was~declared~for~the~latter.
135 }
136
137 \msg_new:nnn { intexgral } { use-glyph-instead }
138 {
139     Consider~using~the~dedicated~glyph~\__intexgral_warning_msg_header:
140     The~key~"nint"~was~used~with~fewer~than~
141     5~regular~integral~signs.
142 }

```

4 Useful variants

```

\str_if_eq:neTF
\str_if_eq:neF
\keys_if_exist:nVTF
\seq_use:Ne
\str_if_eq_p:en
\prop_gput:Nne
\seq_gset_split:cnn
\seq_set_split:Nnf

```

```

143 \cs_generate_variant:Nn \str_if_eq:nnTF { neTF }
144 \cs_generate_variant:Nn \str_if_eq:nnF { neF }
145 \cs_generate_variant:Nn \keys_if_exist:nnTF { nVTF }
146 \cs_generate_variant:Nn \seq_use:Nn { Ne }
147 \cs_generate_variant:Nn \str_if_eq_p:nn { en }
148 \cs_generate_variant:Nn \prop_gput:Nnn { Nne }
149 \cs_generate_variant:Nn \seq_gset_split:Nnn { cnn }
150 \cs_generate_variant:Nn \seq_set_split:Nnn { Nnf }

```

5 Internal variables

5.1 Tokens

```

\l_@@_integrand_tl

```

Token containing the integrand. It is used as-is with the `default` mode, whereas for the other modes, it is subsequently separated into a sequence.

```

151 \tl_new:N \l__intexgral_integrand_tl

```

```

\l_@@_integral_symbol_tl

```

Token containing the integral symbol. When not modified by the `symbol` key, it should still be initialised to `\int`.

```

152 \tl_new:N \l__intexgral_integral_symbol_tl
153 \tl_set_eq:NN \l__intexgral_integral_symbol_tl \int

```

```

\l_@@_lower_limit_tl

```

```

\l_@@_upper_limit_tl

```

Tokens containing the limits.

```

154 \tl_new:N \l__intexgral_lower_limit_tl
155 \tl_new:N \l__intexgral_upper_limit_tl

```

```

\l_@@_differential_symbol_tl

```

Token containing the symbol for differentials, which can be modified with the `diff-symb` key. It is initialised to the `\l_@@_default_differential_symbol_tl` token.

```

156 \tl_new:N \l__intexgral_differential_symbol_tl

```

```

\l_@@_vectorial_differential_style_tl

```

```

\l_@@_domain_style_tl

```

Tokens containing the macros to apply to the `diff-vec` and `domain(*)` keys.

```

157 \tl_new:N \l__intexgral_vectorial_differential_style_tl
158 \tl_new:N \l__intexgral_domain_style_tl

```

`\l_@@_domain_char_tl`
`\l_@@_domain_dimension_tl`

Token used for storing respectively the character and the dimension of a domain.

```
159 \tl_new:N \l__intexgral_domain_char_tl
160 \tl_new:N \l__intexgral_domain_dimension_tl
```

`\c_@@_left_bracket_tl`
`\c_@@_right_bracket_tl`

Constant tokens for the `limits*` key.

```
161 \tl_const:Nn \c__intexgral_left_bracket_tl { [ }
162 \tl_const:Nn \c__intexgral_right_bracket_tl { ] }
```

`\l_@@_key_name_tl`

Token containing the potential name of a key, used to differentiate the syntax `<key=value>` from the special syntax.

```
163 \tl_new:N \l__intexgral_key_name_tl
```

5.2 Booleans

`\l_@@_manual_differential_bool`

Boolean used to determine if the differentials are inserted through `\differentials`.

```
164 \bool_new:N \l__intexgral_manual_differential_bool
```

`\l_@@_custom_variables_bool`

Boolean used to determine if the differentials are inserted through the `variables` key.

```
165 \bool_new:N \l__intexgral_custom_variables_bool
```

`\l_@@_separate_integral_bool`

Boolean used to determine if the integral is *separated* into several parts, that is to say, if the modes `nested` or `product` are in effect.

```
166 \bool_new:N \l__intexgral_separate_integral_bool
```

`\l_@@_vectorial_differential_bool`

Boolean used to determine if vectors should be applied to the differentials.

```
167 \bool_new:N \l__intexgral_vectorial_differential_bool
```

`\l_@@_jacobian_bool`

Boolean used to determine if the Jacobian should be displayed.

```
168 \bool_new:N \l__intexgral_jacobian_bool
```

5.3 Sequences and clists

`\l_@@_domain_seq`

Sequence containing the integration domains and their dimensions (the main sequences will be explained later).

```
169 \seq_new:N \l__intexgral_domain_seq
```

`\l__intexgral_default_variables_seq`
`\l__intexgral_default_vectorial_variables_seq`

Sequences containing the default variables, used when the `variables` key is not specified.

```
170 \seq_new:N \l__intexgral_default_variables_seq
171 \seq_new:N \l__intexgral_default_vectorial_variables_seq
```

`\l_@@_differential_order_clist`

List containing the powers of the differentials (clist and not sequence since `.seq_set:N` does not exist for keys).

```
172 \clist_new:N \l__intexgral_differential_order_clist
```

5.4 Property lists

`\g_@@_limits_keyword_prop`
`\g_@@_differential_group_keyword_prop`

A property list is created to store limits and variable keywords.

```
173 \prop_new:N \g__intexgral_limits_keyword_prop
174 \prop_new:N \g__intexgral_differential_group_keyword_prop
```

5.5 Muskip

`\l_@@_symbol_inner_sep_muskip`
`\l_@@_symbol_post_sep_muskip`
`\l_@@_differential_sep_muskip`
`\l_@@_variable_sep_muskip`

Internal muskips used to store the spacings for respectively:

1. The spacing between the symbols when several are generated by `limits(*)`.
2. The spacing that follows the symbol.
3. The spacing between the differentials when the key `variables` is used.
4. The spacing between the differentials and the integrand.

```
175 \muskip_new:N \l__intexgral_symbol_inner_sep_muskip
176 \muskip_new:N \l__intexgral_symbol_post_sep_muskip
177 \muskip_new:N \l__intexgral_variable_sep_muskip
178 \muskip_new:N \l__intexgral_differential_sep_muskip
```

5.6 String

`\l_@@_display_mode_str`

Internal variable used to store the integral's display mode. The reason for using a *string* stems from the lack of a `\tl_case:nnTF` macro.

```
179 \str_new:N \l__intexgral_display_mode_str
```

6 Auxiliary macros and sequences

Here we define the two $\TeX$ primitives `\mkern` and `\mathchoice` with an *expl3*-like syntax.

```
\@@_mkern:N \@@_mkern:N <muskip>
```

```
180 \cs_new_protected:Npn \__intexgral_mkern:N #1
181   { \tex_mkern:D #1 \scan_stop: }
```

```
\@@_mathchoice:nnnn \@@_mathchoice:nnnn <{<display style>}> <{<inline style>}> <{<script style>}>
  <{<scriptscript style>}>
```

```
182 \cs_new_eq:NN \__intexgral_mathchoice:nnnn \mathchoice
```

```
\@@_invert_limits:w \@@_invert_limits:w <lower limit>,<upper limit>\q_stop
```

Macro inverting the limits for the `invert-limits` option. The action is fully expandable and is more efficient than a `\seq_inverse:N` in the context of this package, knowing that a maximum of two elements are present.

```
183 \cs_new:Npn \__intexgral_invert_limits:w #1,#2\q_stop{#2,#1}
```

```
\@@_general_integral_symbol:
```

We create a macro that contains the general expression for an integral: symbol, limits style, lower limit and upper limit. We just have to assign the tokens to compose a complete symbol, and possibly repeat the macro for integrals defined on several variables.

```
184 \cs_new:Npn \__intexgral_general_integral_symbol:
185   {
186     \l__intexgral_integral_symbol_tl
187     \l__intexgral_limits_style_tl
188     \c_math_subscript_token
189     { \l__intexgral_lower_limit_tl }
190     \c_math_superscript_token
191     { \l__intexgral_upper_limit_tl }
192   }
```

6.1 Main sequences

Given the large number of existing keys and possible combinations, the tokens must be carefully organized and stored in order to assemble the final integral. The approach taken was to create sequences dedicated to each element of an integral:

```
\l_@@_integral_symbol_seq
```

One for the symbol (or the *symbols*, if the key `limits*` is in use).

```
193 \seq_new:N \l__intexgral_integral_symbol_seq
```

`\l_@@_integrand_seq`

One for the integrand (or *the* integrands, if split into several parts by `nested` or `product`).

194 `\seq_new:N \l__intexgral_integrand_seq`

`\l_@@_jacobian_seq`

One for the Jacobian.

195 `\seq_new:N \l__intexgral_jacobian_seq`

`\l_@@_differential_seq`

And finally, one for the differentials.

196 `\seq_new:N \l__intexgral_differential_seq`

This way, we can construct the final integral by varying how we extract the different elements from these sequences. Here is a more concrete example:

For an indefinite integral.

$$\overset{\text{S}}{\boxed{\iint_S}} \overset{\text{I}}{\boxed{f(x,y)}} \overset{\text{D}}{\boxed{dx dy}}$$

For a definite integral.

$$\overset{\text{S}_1}{\boxed{\int_a^b}} \overset{\text{S}_2}{\boxed{\int_c^d}} \overset{\text{I}_1}{\boxed{f(x)}} \overset{\text{I}_2}{\boxed{g(y)}} \overset{\text{D}_1}{\boxed{dx}} \overset{\text{D}_2}{\boxed{dy}}$$

6.2 Supplementary sequences

A few additional sequences will be required during the intermediate stages, which will involve preparing the four main sequences listed above.

`\l_@@_variables_seq`

Firstly, a sequence containing the variables.

197 `\seq_new:N \l__intexgral_variables_seq`

`\l_@@_limits_seq`

Then, a sequence containing the pairs of integration limits, that is to say the elements of the `limits` key separated in two by the semicolon.

198 `\seq_new:N \l__intexgral_limits_seq`

`\l_@@_upper_limits_seq`

`\l_@@_lower_limits_seq`

Two sequences that contain the upper and lower limits respectively. Thus, if the key `limits` contains the elements `a;b, c;d` and `e;f`, then the sequence of lower limits will contain the elements `a, c` and `e`, while the sequence of upper limits will contain the elements `b, d` and `f`.

199 `\seq_new:N \l__intexgral_upper_limits_seq`

200 `\seq_new:N \l__intexgral_lower_limits_seq`

7 Integration limits

\@@_has_pm_p:n

We define a conditional macro that will handle the `+-<limit>` syntax. It simply checks that the first and second tokens of the `limits` key argument correspond respectively to `+` and `-`.

```

201 \prg_new_conditional:Npnn \__intexgral_has_pm:n #1 { p }
202 {
203     \bool_lazy_and:nnTF
204     { \exp_args:Ne \str_if_eq_p:nn { \str_item:nn { #1 } { 1 } } { + } }
205     { \exp_args:Ne \str_if_eq_p:nn { \str_item:nn { #1 } { 2 } } { - } }
206     { \prg_return_true: }
207     { \prg_return_false: }
208 }
```

\@@_trim_pm:w

Then, a delimited macro is created to remove the `+-`

```

209 \cs_new:Npn \__intexgral_trim_pm:w +-#1\q_stop{#1}
```

\@@_parse_limits:n ★

`\@@_parse_limits:n {<lower limit>}{< upper limit>}, <keyword>`

This macro executes one of the following actions depending on the boolean value that is true:

- If the argument is a keyword defined for limits, the macro expands to its content, inverting it if necessary (depending on the status of `\l_@@_invert_limits_bool`).
- If the argument starts with the tokens `+-`, removes them and adds the signs manually.
- If none of these two conditions are met, simply leaves the argument in the input stream.

```

210 \cs_new:Npn \__intexgral_parse_limits:n #1
211 {
212     \bool_case:nF
213     {
214         { \prop_if_in_p:Nn \g__intexgral_limits_keyword_prop { #1 } }
215         {
216             \bool_if:NTF \l__intexgral_invert_limits_bool
217             {
218                 \exp_last_unbraced:Ne \__intexgral_invert_limits:w
219                 { \prop_item:Nn \g__intexgral_limits_keyword_prop { #1 } }
220                 \q_stop
221             }
222             { \prop_item:Nn \g__intexgral_limits_keyword_prop { #1 } }
223         }
224         { \__intexgral_has_pm_p:n { #1 } }
225         {
226             -\__intexgral_trim_pm:w #1\q_stop,
227             +\__intexgral_trim_pm:w #1\q_stop
228         }
229     }
230     { #1 }
231 }
```

`\@@_set_limits_regular:`

Once the argument of the key `limits` has been converted into a sequence (action performed within `.code:n` in the key declaration), each pair of limits is assigned to a temporary sequence. This sequence is created so that if a keyword is present, it is replaced by its corresponding limits according to the previous macro. The action is executed within a `\romannumeral`-type expansion to avoid expanding limits containing fragile macros (such as `\mathbb`). The lower and upper limits can then be extracted from this sequence and assigned to their respective sequences.

```
232 \cs_new_protected:Nn \__intexgral_set_limits_regular:
233 {
234   \seq_map_inline:Nn \l__intexgral_limits_seq
235   {
236     \seq_set_split:Nnf \l_tmpa_seq { , }
237     { \__intexgral_parse_limits:n { ##1 } }
238     \seq_put_right:Ne \l__intexgral_lower_limits_seq
239     { \seq_item:Nn \l_tmpa_seq { 1 } }
240     \seq_put_right:Ne \l__intexgral_upper_limits_seq
241     { \seq_item:Nn \l_tmpa_seq { 2 } }
242   }
243 }
```

`\@@_set_limits_starred:`

The following macro is similar to the previous one, but it handles limits in the form of intervals when the key `limits*` is used.

```
244 \cs_new_protected:Npn \__intexgral_set_limits_starred:
245 {
246   \seq_map_indexed_inline:Nn \l__intexgral_limits_seq
247   {
248     \seq_set_split:Nnf \l_tmpa_seq { , }
249     { \__intexgral_parse_limits:n { ##2 } }
250     \bool_if:NTF \l__intexgral_invert_limits_bool
251     {
252       \tl_set:Ne \l__intexgral_lower_limit_tl
253       {
254         \seq_item:Nn \l_tmpa_seq { 2 }
255         \tex_mathpunct:D,
256         \seq_item:Nn \l_tmpa_seq { 1 }
257       }
258     }
259     {
260       \tl_set:Ne \l__intexgral_lower_limit_tl
261       {
262         \seq_item:Nn \l_tmpa_seq { 1 }
263         \tex_mathpunct:D,
264         \seq_item:Nn \l_tmpa_seq { 2 }
265       }
266     }
267     \bool_if:NTF \l__intexgral_invert_limits_bool
268     { \seq_put_right:Ne \l__intexgral_upper_limits_seq }
269     { \seq_put_right:Ne \l__intexgral_lower_limits_seq }
270     {
271       \str_case_e:nnF { \seq_item:Nn \l_tmpa_seq { 1 } }
272       { { -\infty } { \tex_left:D \c__intexgral_right_bracket_tl } }
273       { \tex_left:D \c__intexgral_left_bracket_tl }
```

```

274         \tl_use:N \l__intexgral_lower_limit_tl
275         \str_case_e:nnF { \seq_item:Nn \l_tmpa_seq { 2 } }
276         { { +\infty } { \tex_right:D \c__intexgral_left_bracket_tl } }
277         { \tex_right:D \c__intexgral_right_bracket_tl }
278     }
279 }
280 }

```

\@@_set_limits:nn \@@_set_limits:nn {<lower limit>} {<upper limit>}

According to the `invert-limits` option, this macro assigns the lower and upper limits to their respective tokens. If it is active, the assignment becomes somewhat counterintuitive (the lower limit is assigned to `\l__@@_upper_limit_tl` and vice versa), but it will still produce the expected result. The expansion of the limits is prevented to avoid issues with fragile macros as seen previously.

```

281 \cs_new_protected:Npn \__intexgral_set_limits:nn #1#2
282 {
283     \bool_if:NTF \l__intexgral_invert_limits_bool
284     {
285         \tl_set:Nx \l__intexgral_lower_limit_tl { \exp_not:n { #2 } }
286         \tl_set:Nx \l__intexgral_upper_limit_tl { \exp_not:n { #1 } }
287     }
288     {
289         \tl_set:Nx \l__intexgral_lower_limit_tl { \exp_not:n { #1 } }
290         \tl_set:Nx \l__intexgral_upper_limit_tl { \exp_not:n { #2 } }
291     }
292 }

```

\@@_generate_integral_sequence:

Once the sequences of lower and upper limits are filled, we can use the structure of the sequences to our advantage to automatically generate as many symbols as necessary. We proceed with an incremented loop in which we perform two actions:

1. With the help of `\@@_set_limits:nn`, we assign the integration limits to the `\l__@@_lower_limit_tl` and `\l__@@_upper_limit_tl` tokens from the i^{th} element of the `\l__@@_lower_limits_seq` and `\l__@@_upper_limits_seq` sequences respectively.
2. We place to the right (i.e. in order) of the `\l__@@_integral_symbol_seq` sequence the `\@@_general_integral_symbol:` macro, which contains the general form of an integral. Note that we fully expand its content in order to retrieve the immediate assignment of tokens.

```

293 \cs_new_protected:Nn \__intexgral_generate_integral_sequence:
294 {
295     \int_set:Nn \l_tmpa_int
296     {
297         \seq_if_empty:NTF \l__intexgral_lower_limits_seq
298         { \seq_count:N \l__intexgral_upper_limits_seq }
299         { \seq_count:N \l__intexgral_lower_limits_seq }
300     }
301     \int_step_inline:nn { \l_tmpa_int }
302     {

```

```

303         \__intexgral_set_limits:nn
304         { \seq_item:Nn \l__intexgral_lower_limits_seq { ##1 } }
305         { \seq_item:Nn \l__intexgral_upper_limits_seq { ##1 } }
306     \seq_put_right:Ne \l__intexgral_integral_symbol_seq
307     { \__intexgral_general_integral_symbol: }
308 }
309 }

```

8 Variables

`\@@_parse_variables:n` `\@@_parse_variables:n` $\{ \langle \text{integration variable} \rangle \}$, $\langle \text{key} \rangle$

This macro is applied to each element of the argument of the `variables` key to check whether it is a key or an explicit list, in the same way as for the limits. It mainly serves to transfer the elements of `\l_@@_variables_seq` (containing for example “ x, y, z ”) to `\l_@@_differential_seq` (which will contain “ dx, dy, dz ”); while taking into account the different options that have been applied. In the case where no variable is entered, the macro takes care of placing x or $\vec{r}$ depending on the value of the boolean `\l_@@_vectorial_differential_bool`.

```

310 \cs_new_protected:Nn \__intexgral_parse_variables:
311 {
312     \bool_if:NF \l__intexgral_custom_variables_bool
313     {
314         \bool_if:NTF \l__intexgral_vectorial_differential_bool
315         {
316             \seq_set_eq:NN
317             \l__intexgral_variables_seq
318             \l__intexgral_default_vectorial_variables_seq
319         }
320         {
321             \seq_set_eq:NN
322             \l__intexgral_variables_seq
323             \l__intexgral_default_variables_seq
324         }
325     }
326     \seq_map_indexed_inline:Nn \l__intexgral_variables_seq
327     {
328         \seq_put_right:Ne \l__intexgral_differential_seq
329         {
330             \exp_not:V \l__intexgral_differential_symbol_tl
331             \clist_if_empty:NF \l__intexgral_differential_order_clist
332             {
333                 \c_math_superscript_token
334                 {
335                     \clist_item:Nn
336                     \l__intexgral_differential_order_clist
337                     { ##1 }
338                 }
339             }
340             \bool_if:NT \l__intexgral_vectorial_differential_bool
341             { \exp_not:V \l__intexgral_vectorial_differential_style_tl }
342             { ##2 }
343         }
344     }
345 }

```

9 Special syntax

```
\@@_retrieve_key_name:w ★ \@@_retrieve_key_name:w <clé>=<valeur> \q_stop
```

In order to handle the special syntax, we start by creating a delimited macro that extracts the key name and its value, and that only returns the former.

```
346 \cs_new:Npn \__intexgral_retrieve_key_name:w #1=#2\q_stop { #1 }
```

```
\@@_extract_first_key_name:n \@@_extract_first_key_name:n {<\integral's optional argument>}
```

We then associate the optional argument received by `\integral` to a temporary *comma list*, from which we extract the first element; we then check if it contains an “=” sign. If so, we use the previous macro to retrieve only the key name; otherwise, we simply copy the group of tokens retrieved into `\l_@@_key_name_tl`.

```
347 \cs_new_protected:Npn \__intexgral_extract_first_key_name:n #1
348 {
349   \seq_set_split:Nnn \l_tmpa_seq { : } { #1 }
350   \tl_set:Nc \l_tmpa_tl { \seq_item:Nn \l_tmpa_seq { 1 } }
351   \tl_if_in:NnTF \l_tmpa_tl { = }
352   {
353     \tl_set:Nc \l__intexgral_key_name_tl
354     {
355       \exp_last_unbraced:NV
356       \__intexgral_retrieve_key_name:w \l_tmpa_tl \q_stop
357     }
358   }
359   { \tl_set_eq:NN \l__intexgral_key_name_tl \l_tmpa_tl }
360 }
```

```
\@@_parse_integral_keys:n \@@_parse_integral_keys:n {<first key of the optional argument>}
```

Using the `l3keys` module, we can check if the captured group of tokens corresponds to a defined key, in which case we directly apply `\keys_set:nn`. If the test is negative, we manually assign the keys by considering the optional argument as a sequence to be delimited with a colon. Temporary sequences are used to manipulate the different parts of the argument. We notably check for the presence of the token `+j` to activate the `jacobian` key, and we add the default variable if it is not specified. Finally, we assign values to the `limits`, `variables` and `mode` keys based on the extracted elements.

```
361 \cs_new_protected:Npn \__intexgral_parse_integral_keys:n #1
362 {
363   \keys_if_exist:nVTF { integral } \l__intexgral_key_name_tl
364   { \keys_set:nn { integral } { #1 } }
365   {
366     \regex_split:nnN { : } { #1 } \l_tmpa_seq
367     \str_if_eq:eeTF
368     { \seq_item:Nn \l_tmpa_seq { 2 } }
369     { +j }
370     {
371       \exp_args:NNne \seq_set_item:Nnn \l_tmpa_seq { 2 }
372       {
373         \seq_use:Nn
374         \l__intexgral_default_variables_seq
375         { , }
376         +j

```

```

377         }
378     }
379     {
380         \int_compare:nNnT { \seq_count:N \l_tmpa_seq } < { 2 }
381         {
382             \seq_put_right:Ne
383             \l_tmpa_seq
384             { \seq_use:Nn \l__intexgral_default_variables_seq { , } }
385         }
386     }
387     \int_compare:nNnT { \seq_count:N \l_tmpa_seq } < { 3 }
388     { \seq_put_right:Nn \l_tmpa_seq { d } }
389     \seq_set_split:Nne \l_tmpb_seq
390     { + }
391     { \seq_item:Nn \l_tmpa_seq { 2 } }
392     \keys_set:ne { integral }
393     {
394         limits      = { \seq_item:Nn \l_tmpa_seq { 1 } },
395         variables   = { \seq_item:Nn \l_tmpb_seq { 1 } },
396         mode        =
397         {
398             \str_case:enF { \seq_item:Nn \l_tmpa_seq { 3 } }
399             {
400                 { d      } { default }
401                 { n      } { nested  }
402                 { p      } { product }
403                 { default } { default }
404                 { nested } { nested  }
405                 { product } { product }
406             }
407             { default }
408         },
409         \bool_if:nT
410         {
411             \int_compare_p:nNn { \seq_count:N \l_tmpb_seq } > { 1 }
412             &&
413             \str_if_eq_p:en { \seq_item:Nn \l_tmpb_seq { 2 } } { j }
414         }
415         { jacobian }
416     }
417 }
418 }

```

10 Typesetting the integral

`\@@_integral_preconfiguration:`

Before being able to typeset the integral in the document, a number of checks must be performed. Indeed, depending on the combination of keys specified in the optional argument, the sequences may not be filled yet. We therefore perform the following checks:

- We check if the integrand should be split into several parts by the `nested` and `product` modes, in which case we populate the integrand sequence by splitting the elements at the semicolon. Otherwise, we simply add the integrand (`\l_@@_integrand_tl`) to the sequence.
- We verify if the sequence of symbols is empty, indicating that the `limits*` key has not been called. Indeed, only the use of this key triggers the execution of all macros presented in section 7. In this case, we simply add to the sequence the general form of the integral, whose symbol and bounds will be optionally modified by the keys `symbol`, `ulimit` and `llimit`.
- We then control the deactivation of differentials in addition to detecting the presence of `\differentials` using a regular expression.

```

419 \cs_new_protected:Nn \__intexgral_integral_preconfiguration:
420 {
421     \bool_if:NTF \l__intexgral_separate_integral_bool
422     {
423         \seq_set_split:NnV
424             \l__intexgral_integrand_seq
425             { ; }
426             \l__intexgral_integrand_tl
427     }
428     {
429         \seq_put_right:NV
430             \l__intexgral_integrand_seq
431             \l__intexgral_integrand_tl
432     }
433     \seq_if_empty:NT \l__intexgral_integral_symbol_seq
434     {
435         \seq_put_right:Nn \l__intexgral_integral_symbol_seq
436             { \__intexgral_general_integral_symbol: }
437     }
438     \bool_if:NF \l__intexgral_deactivate_variables_bool
439     { \__intexgral_parse_variables: }
440     \tl_if_regex_match:VnTF \l__intexgral_integrand_tl { \c{differentials} }
441     { \bool_set_true:N \l__intexgral_manual_differential_bool }
442     { \bool_set_false:N \l__intexgral_manual_differential_bool }
443 }
```

We can now define the six macros that will be used to typeset a precise integral; three modes allowed by `mode`, each accompanied by a variant for the inverted differentials. It should also be noted that `\differentials` is only created if the boolean `\l_@@_manual_differential_bool` is true, in order to avoid unnecessary definitions.

10.1 Mode *default*

`\@@_print_default_integral:`

For the default mode, we typeset in the following order: integral symbol, integrand, Jacobian (if applicable), and differentials. Since the elements are successive, we apply `\seq_use:Nn` for each of the sequences, inserting a kern when necessary. Note that if `\l_@@_vectorial_differential_bool` is true, we add a middle dot and remove the previous kern (for spacing reasons related to `\cdot`).

```
444 \cs_new_protected:Npn \__intexgral_print_default_integral:
445 {
446     \bool_if:NT \l__intexgral_manual_differential_bool
447     {
448         \cs_set_protected:Npn \differentials
449         {
450             \seq_use:Nn \l__intexgral_differential_seq
451             { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
452         }
453     }
454     \seq_use:Nn \l__intexgral_integral_symbol_seq
455     { \__intexgral_mkern:N \l__intexgral_symbol_inner_sep_muskip }
456     \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip
457     \seq_use:Nn \l__intexgral_integrand_seq { }
458     \bool_if:NT \l__intexgral_jacobian_bool
459     { \seq_use:Nn \l__intexgral_jacobian_seq { } }
460     \__intexgral_mkern:N \l__intexgral_variable_sep_muskip
461     \bool_if:NT \l__intexgral_vectorial_differential_bool
462     { \tex_unkern:D \cdot}
463     \bool_if:NF \l__intexgral_manual_differential_bool
464     {
465         \seq_use:Nn \l__intexgral_differential_seq
466         { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
467     }
468 }
```

`\@@_print_default_integral_inv:`

For the inverted differentials, nothing is very different, except that `\l_@@_integrand_seq` and `\l_@@_differential_seq` are swapped.

```
469 \cs_new_protected:Npn \__intexgral_print_default_integral_inv:
470 {
471     \bool_if:NT \l__intexgral_manual_differential_bool
472     {
473         \cs_set_protected:Npn \differentials
474         {
475             \seq_use:Nn \l__intexgral_differential_seq
476             { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
477         }
478     }
479     \seq_use:Nn \l__intexgral_integral_symbol_seq
480     { \__intexgral_mkern:N \l__intexgral_symbol_inner_sep_muskip }
481     \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip
482     \bool_if:NF \l__intexgral_manual_differential_bool
483     {
484         \seq_use:Nn \l__intexgral_differential_seq
485         { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
```

```

486     }
487     \__intexgral_mkern:N \l__intexgral_variable_sep_muskip
488     \bool_if:NT \l__intexgral_vectorial_differential_bool
489     { \tex_unkern:D \cdot}
490     \seq_use:Nn \l__intexgral_integrand_seq { }
491     \bool_if:NT \l__intexgral_jacobian_bool
492     { \seq_use:Nn \l__intexgral_jacobian_seq { } }
493 }

```

10.2 Mode *nested*

\@@_print_nested_integral:

In the nested mode, we initiate an incremented loop where we extract the i^{th} element of each of the sequences of symbol, integrand and Jacobian. After the iteration, we compose all the differentials again using `\seq_use:Nn`.

```

494 \cs_new_protected:Npn \__intexgral_print_nested_integral:
495 {
496     \bool_if:NT \l__intexgral_manual_differential_bool
497     {
498         \cs_set_protected:Npn \differentials
499         {
500             \seq_use:Nn \l__intexgral_differential_seq
501             { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
502         }
503     }
504     \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
505     {
506         \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
507         \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip
508         \seq_item:Nn \l__intexgral_integrand_seq { ##1 }
509         \bool_if:NT \l__intexgral_jacobian_bool
510         { \seq_item:Nn \l__intexgral_jacobian_seq { ##1 } }
511     }
512     \__intexgral_mkern:N \l__intexgral_variable_sep_muskip
513     \bool_if:NF \l__intexgral_manual_differential_bool
514     {
515         \seq_use:Nn \l__intexgral_differential_seq
516         { \__intexgral_mkern:N \l__intexgral_differential_sep_muskip }
517     }
518 }

```

\@@_print_nested_integral_inv:

Here, it is the sequence of differentials that appears in the loop and the sequence of the integrand that is used at the end, after the iteration.

```

519 \cs_new_protected:Npn \__intexgral_print_nested_integral_inv:
520 {
521     \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
522     {
523         \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
524         \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip
525         \seq_item:Nn \l__intexgral_differential_seq { ##1 }
526     }
527     \__intexgral_mkern:N \l__intexgral_variable_sep_muskip

```



```

528     \seq_use:Nn \l__intexgral_integrand_seq { }
529     \bool_if:NT \l__intexgral_jacobian_bool
530     { \seq_use:Nn \l__intexgral_jacobian_seq { } }
531 }

```

10.3 Mode *product*

\@@_print_product_integral:

The product mode is very similar to the nested mode, in the sense that all sequences appear in the loop. Unlike the two previous modes, `\differentials` is defined differently this time: it extracts the elements of `\l__@@_differential_seq` in order and then displays them. The extraction is global in case the macro is used within a group.

```

532 \cs_new_protected:Npn \__intexgral_print_product_integral:
533 {
534     \bool_if:NT \l__intexgral_manual_differential_bool
535     {
536         \cs_set_protected:Npn \differentials
537         {
538             \seq_gpop_left:NN \l__intexgral_differential_seq \l_tmpa_tl
539             \tl_use:N \l_tmpa_tl
540         }
541     }
542     \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
543     {
544         \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
545         \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip
546         \seq_item:Nn \l__intexgral_integrand_seq { ##1 }
547         \bool_if:NT \l__intexgral_jacobian_bool
548         { \seq_item:Nn \l__intexgral_jacobian_seq { ##1 } }
549         \__intexgral_mkern:N \l__intexgral_variable_sep_muskip
550         \bool_if:NF \l__intexgral_manual_differential_bool
551         { \seq_item:Nn \l__intexgral_differential_seq { ##1 } }
552     }
553 }

```

\@@_print_product_integral_inv:

Similarly, with a few adjustments.

```

554 \cs_new_protected:Npn \__intexgral_print_product_integral_inv:
555 {
556     \bool_if:NT \l__intexgral_manual_differential_bool
557     {
558         \cs_set_protected:Npn \differentials
559         {
560             \seq_gpop_left:NN \l__intexgral_differential_seq \l_tmpa_tl
561             \tl_use:N \l_tmpa_tl
562         }
563     }
564     \int_step_inline:nn { \seq_count:N \l__intexgral_integral_symbol_seq }
565     {
566         \seq_item:Nn \l__intexgral_integral_symbol_seq { ##1 }
567         \bool_if:NF \l__intexgral_manual_differential_bool
568         {
569             \__intexgral_mkern:N \l__intexgral_symbol_post_sep_muskip

```

```

570         \seq_item:Nn \l__intexgral_differential_seq { ##1 }
571     }
572     \__intexgral_mkern:N \l__intexgral_variable_sep_muskip
573     \seq_item:Nn \l__intexgral_integrand_seq { ##1 }
574     \bool_if:NT \l__intexgral_jacobian_bool
575         { \seq_item:Nn \l__intexgral_jacobian_seq { ##1 } }
576     }
577 }

```

10.4 Final typesetting

\@@_print_integral:

The main macro typesetting the integral first increments the global counter allowing each integral to be numbered. Then, it carries out the preparatory actions explained previously. Finally, depending on the placement of the differentials and the requested display mode, one of the six macros is called.

```

578 \cs_new_protected:Nn \__intexgral_print_integral:
579 {
580     \int_gincr:N \g__intexgral_integral_number_int
581     \__intexgral_integral_preconfiguration:
582     \bool_if:NTF \l__intexgral_invert_differential_bool
583     {
584         \str_case:VnF \l__intexgral_display_mode_str
585         {
586             { default } { \__intexgral_print_default_integral_inv: }
587             { nested } { \__intexgral_print_nested_integral_inv: }
588             { product } { \__intexgral_print_product_integral_inv: }
589         }
590         { \__intexgral_print_default_integral_inv: }
591     }
592     {
593         \str_case:VnF \l__intexgral_display_mode_str
594         {
595             { default } { \__intexgral_print_default_integral: }
596             { nested } { \__intexgral_print_nested_integral: }
597             { product } { \__intexgral_print_product_integral: }
598         }
599         { \__intexgral_print_default_integral: }
600     }
601 }

```

11 Key declaration

The creation of keys for `\integral` and `\IntegralSetup` is relatively trivial. The few elementary actions for the keys `limits` and `variables` that were not covered by the macros are performed here.

```

602 \keys_define:nn { integral }
603 {
604     mode .choice:,
605     mode / default .code:n =
606     {
607         \bool_set_false:N \l__intexgral_separate_integral_bool
608         \str_set:Nn \l__intexgral_display_mode_str { default }

```

```

609     },
610     mode / nested .code:n =
611     {
612         \bool_set_true:N \l__intexgral_separate_integral_bool
613         \str_set:Nn \l__intexgral_display_mode_str { nested }
614     },
615     mode / product .code:n =
616     {
617         \bool_set_true:N \l__intexgral_separate_integral_bool
618         \str_set:Nn \l__intexgral_display_mode_str { product }
619     },
620     mode .default:n = { default },
621
622     limits-mode .choice:,
623     limits-mode / limits .code:n =
624         { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_limits:D },
625     limits-mode / nolimits .code:n =
626         { \tl_set_eq:NN \l__intexgral_limits_style_tl \tex_nolimits:D },
627
628     limits .code:n =
629     {
630         \seq_set_split:Nnn \l__intexgral_limits_seq { ; } { #1 }
631         \__intexgral_set_limits_regular:
632         \__intexgral_generate_integral_sequence:
633     },
634
635     limits* .code:n =
636     {
637         \seq_set_split:Nnn \l__intexgral_limits_seq { ; } { #1 }
638         \__intexgral_set_limits_starred:
639         \__intexgral_generate_integral_sequence:
640     },
641
642     llimit .tl_set:N = \l__intexgral_lower_limit_tl,
643
644     ulimit .tl_set:N = \l__intexgral_upper_limit_tl,
645
646     symbol .code:n =
647     {
648         \cs_if_exist:NTF #1
649         { \tl_set_eq:NN \l__intexgral_integral_symbol_tl #1 }
650         {
651             \msg_warning:nnn { intexgral } { unknown-symb } { #1 }
652             \tl_set_eq:NN \l__intexgral_integral_symbol_tl \int
653         }
654     },
655
656     nint .code:n =
657     {
658         \int_compare:nNnT { #1 } < { 5 }
659         { \msg_warning:nn { intexgral } { use-glyph-instead } }
660         \tl_clear:N \l__intexgral_integral_symbol_tl
661         \int_step_inline:nn { #1 }
662         {
663             \tl_put_right:Nn \l__intexgral_integral_symbol_tl
664             { \int }
665             \int_compare:nNnT { ##1 } < { #1 }
666             {

```

```

667         \tl_put_right:Nn \l__intexgral_integral_symbol_tl
668         {
669             \__intexgral_mathchoice:nnnn
670             { \tex_mkern:D -12mu \scan_stop: }
671             { \tex_mkern:D -8mu \scan_stop: }
672             { \tex_mkern:D -4mu \scan_stop: }
673             { \tex_mkern:D -2mu \scan_stop: }
674         }
675     }
676 },
677
678 domain .code:n =
679 {
680     \tl_set:Nn \l_tmpa_tl { #1 }
681     \seq_set_split:NnV \l__intexgral_domain_seq { * } \l_tmpa_tl
682     \seq_map_inline:Nn \l__intexgral_domain_seq
683     {
684         \tl_if_empty:NF \l__intexgral_lower_limit_tl
685         { \tl_put_right:Nn \l__intexgral_lower_limit_tl { \times } }
686         \tl_set:Ne \l__intexgral_domain_char_tl
687         { \exp_args:Ne \str_uppercase:n { \tl_head:n { ##1 } } }
688         \tl_set:Ne \l__intexgral_domain_dimension_tl
689         { \tl_tail:n { ##1 } }
690         \tl_put_right:Ne \l__intexgral_lower_limit_tl
691         {
692             \exp_not:V \l__intexgral_domain_style_tl
693             { \l__intexgral_domain_char_tl }
694             \c_math_superscript_token { \l__intexgral_domain_dimension_tl }
695         }
696     }
697 },
698
699 domain* .code:n =
700 {
701     \tl_set:Nn \l_tmpa_tl { #1 }
702     \seq_set_split:NnV \l__intexgral_domain_seq { * } \l_tmpa_tl
703     \seq_map_inline:Nn \l__intexgral_domain_seq
704     {
705         \tl_if_empty:NF \l__intexgral_lower_limit_tl
706         { \tl_put_right:Nn \l__intexgral_lower_limit_tl { \times } }
707         \tl_set:Ne \l__intexgral_domain_char_tl
708         { \exp_args:Ne \str_uppercase:n { \tl_head:n { ##1 } } }
709         \tl_set:Ne \l__intexgral_domain_dimension_tl
710         { \tl_tail:n { ##1 } }
711         \tl_put_right:Ne \l__intexgral_lower_limit_tl
712         {
713             \exp_not:V \l__intexgral_domain_style_tl
714             { \l__intexgral_domain_char_tl }
715             \c_math_subscript_token { \l__intexgral_domain_dimension_tl }
716         }
717     }
718 },
719
720 boundary .code:n =
721 { \tl_set:Nn \l__intexgral_lower_limit_tl { \partial #1 } },
722
723 variables .code:n =

```

```

725 {
726   \bool_set_true:N \l__intexgral_custom_variables_bool
727   \str_if_eq:nnTF { #1 } { none }
728   { \bool_set_true:N \l__intexgral_deactivate_variables_bool }
729   {
730     \prop_get:NnNTF
731     \g__intexgral_differential_group_keyword_prop { #1 } \l_tmpa_tl
732     {
733       \seq_set_split:NnV
734       \l__intexgral_variables_seq
735       { , }
736       \l_tmpa_tl
737       \seq_if_exist:cTF { g__intexgral_#1_jacobian_seq }
738       {
739         \seq_set_eq:Nc
740         \l__intexgral_jacobian_seq
741         { g__intexgral_#1_jacobian_seq }
742       }
743       { \msg_warning:nnn { intexgral } { no-jacobian } { #1 } }
744     }
745     { \seq_set_split:Nnn \l__intexgral_variables_seq { , } { #1 } }
746   }
747 },
748
749 jacobian .code:n =
750 { \bool_set_true:N \l__intexgral_jacobian_bool },
751 diff-vec .code:n =
752 { \bool_set_true:N \l__intexgral_vectorial_differential_bool },
753 diff-symb .tl_set:N = \l__intexgral_differential_symbol_tl,
754 diff-order .clist_set:N = \l__intexgral_differential_order_clist,
755 }
756
757 \keys_define:nn { IntegralSetup }
758 {
759   diff-symb .tl_set:N = \l__intexgral_differential_symbol_tl,
760   defaultvar .code:n =
761   {
762     \seq_set_split:Nnn
763     \l__intexgral_default_variables_seq
764     { , } { #1 }
765   },
766   defaultvar* .code:n =
767   {
768     \seq_set_split:Nnn
769     \l__intexgral_default_vectorial_variables_seq
770     { , } { #1 }
771   },
772   postsymbsep .muskip_set:N = \l__intexgral_symbol_post_sep_muskip,
773   varsep .muskip_set:N = \l__intexgral_variable_sep_muskip,
774   diffsep .muskip_set:N = \l__intexgral_differential_sep_muskip,
775   vectorstyle .tl_set:N = \l__intexgral_vectorial_differential_style_tl,
776   domainstyle .tl_set:N = \l__intexgral_domain_style_tl,
777   innersymbsep .tl_set:N = \l__intexgral_symbol_inner_sep_muskip,
778   novar .bool_set:N = \l__intexgral_deactivate_variables_bool,
779 }

```

12 User-level macros

12.1 Keywords for limits

`\@@_new_limits_group:nn` `\@@_new_limits_group:nn` `{\keyword}` `{\limits}`

For the functioning of `\NewLimitsKeyword`, we register the keyword and its value in a property list. A particularity that nevertheless needs to be explained is that there is a double inversion of the limits: at the time of the creation of the keyword, and at the time of extracting its content (cf. `\@@_parse_limits:nn`). The reason is that it is difficult to attach to a keyword the status of *inverted limits*. Thus, whatever convention is chosen, the limits are stored *in the same order*. This also allows the keywords defined by the package to be in the correct order, even if the convention is changed after loading it.

```
780 \cs_new_protected:Npn \__intexgral_new_limits_group:nn #1#2
781 {
782     \prop_gput:Nne \g__intexgral_limits_keyword_prop { #1 }
783     {
784         \bool_if:NTF \l__intexgral_invert_limits_bool
785             { \__intexgral_invert_limits:w #2\q_stop }
786             { #2 }
787     }
788 }
```

`\NewLimitsKeyword` *This macro is documented in section 4.2.1.*

```
789 \NewDocumentCommand\NewLimitsKeyword{ m m }
790 {
791     \prop_if_in:NnTF \g__intexgral_limits_keyword_prop { #1 }
792         { \msg_error:nnn { intexgral } { lim-group-alr-def } { #1 } }
793         { \__intexgral_new_limits_group:nn { #1 } { #2 } }
794 }
```

`\RenewLimitsKeyword` *This macro is documented in section 4.2.1.*

```
795 \NewDocumentCommand\RenewLimitsKeyword{ m m }
796 {
797     \prop_pop:NnNTF \g__intexgral_limits_keyword_prop { #1 } \l_tmpa_tl
798         { \__intexgral_new_limits_group:nn { #1 } { #2 } }
799         { \msg_error:nnn { intexgral } { lim-group-not-def } { #1 } }
800 }
```

`\ProvideLimitsKeyword` *This macro is documented in section 4.2.1.*

```
801 \NewDocumentCommand\ProvideLimitsKeyword{ m m }
802 {
803     \prop_if_in:NnF \g__intexgral_limits_keyword_prop { #1 }
804         { \__intexgral_new_limits_group:nn { #1 } { #2 } }
805 }
```

`\DeclareLimitsKeyword`

This macro is documented in section 4.2.1.

```
806 \NewDocumentCommand\DeclareLimitsKeyword{ m m }
807 {
808   \prop_remove:Nn \g__intexgral_limits_keyword_prop { #1 }
809   \__intexgral_new_limits_group:nn { #1 } { #2 }
810 }
```

12.2 Keywords for variables

`\@@_new_variables_group:nnn` `\@@_new_variables_group:nnn` `{\keyword}` `{\variable}` `[\jacobian]`

Just like for the limits, we register the groups of differentials in a property list, with their name, value and eventually their associated jacobian.

```
811 \cs_new_protected:Npn \__intexgral_new_variables_group:nnn #1#2#3
812 {
813   \prop_gput:Nnn \g__intexgral_differential_group_keyword_prop
814     { #1 } { #2 }
815   \tl_if_blank:nF { #3 }
816   {
817     \seq_if_exist:cF { g__intexgral_#1_jacobian_seq }
818     { \seq_new:c { g__intexgral_#1_jacobian_seq } }
819     \seq_gset_split:cnn { g__intexgral_#1_jacobian_seq } { , } { #3 }
820   }
821 }
```

`\NewVariableKeyword`

This macro is documented in section 4.3.1.

```
822 \NewDocumentCommand\NewVariableKeyword{ m m o }
823 {
824   \prop_if_in:NnTF \g__intexgral_differential_group_keyword_prop { #1 }
825     { \msg_error:nnn { intexgral } { diff-group-alr-def } { #1 } }
826     { \__intexgral_new_variables_group:nnn { #1 } { #2 } { #3 } }
827 }
```

`\RenewVariableKeyword`

This macro is documented in section 4.3.1.

```
828 \NewDocumentCommand\RenewVariableKeyword{ m m o }
829 {
830   \prop_pop:NnNTF \g__intexgral_differential_group_keyword_prop { #1 }
831     \l_tmpa_tl
832     {
833       \seq_gclear:c { g__intexgral_#1_jacobian_seq }
834       \__intexgral_new_variables_group:nnn { #1 } { #2 } { #3 }
835     }
836     { \msg_error:nnn { intexgral } { diff-group-not-def } { #1 } }
837 }
```

`\ProvideVariableKeyword`

This macro is documented in section 4.3.1.

```

838 \NewDocumentCommand\ProvideVariableKeyword{ m m o }
839 {
840   \prop_if_in:NnF \g__intexgral_differential_group_keyword_prop { #1 }
841   { \__intexgral_new_variables_group:nnn { #1 } { #2 } { #3 } }
842 }

```

`\DeclareVariableKeyword`

This macro is documented in section 4.3.1.

```

843 \NewDocumentCommand\DeclareVariableKeyword{ m m o }
844 {
845   \prop_remove:Nn \g__intexgral_differential_group_keyword_prop { #1 }
846   \seq_gclear:c { g__intexgral_#1_jacobian_seq }
847   \__intexgral_new_variables_group:nnn { #1 } { #2 } { #3 }
848 }

```

12.3 Keywords for symbols

This section does not require an intermediate macro, since we modify the keys of the *integral* module directly.

`\NewSymbolKeyword`

This macro is documented in section 4.4.

```

849 \NewDocumentCommand\NewSymbolKeyword{ m m }
850 {
851   \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
852   { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
853   \keys_if_exist:nnTF { integral } { #1 }
854   { \msg_error:nnn { intexgral } { symb-key-alr-def } { #1 } }
855   {
856     \keys_define:nn { integral }
857     {
858       #1 .meta:n =
859       {
860         symbol=#2,
861         llimit=##1
862       },
863     }
864   }
865 }

```

`\RenewSymbolKeyword`

This macro is documented in section 4.4.

```

866 \NewDocumentCommand\RenewSymbolKeyword{ m m }
867 {
868   \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
869   { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
870   \keys_if_exist:nnTF { integral } { #1 }
871   {
872     \keys_define:nn { integral }
873     {
874       #1 .undefine:,
875       #1 .meta:n =
876       {

```



```

877             symbol=#2,
878             llimit=##1
879         }
880     }
881 }
882 { \msg_error:nnn { intexgral } { symb-key-not-def } }
883 }

```

\ProvideSymbolKeyword

This macro is documented in section 4.4.

```

884 \NewDocumentCommand\ProvideSymbolKeyword{ m m }
885 {
886     \prop_if_in:NnT \g__intexgral_limits_keyword_prop { #1 }
887     { \msg_warning:nnn { intexgral } { key-exists-for-lim } { #1 } }
888     \keys_if_exist:nnF { integral } { #1 }
889     {
890         \keys_define:nn { integral }
891         {
892             #1 .meta:n =
893             {
894                 symbol=#2,
895                 llimit=##1
896             },
897         }
898     }
899 }

```

\DeclareSymbolKeyword

This macro is documented in section 4.4.

```

900 \NewDocumentCommand\DeclareSymbolKeyword{ m m }
901 {
902     \keys_define:nn { integral }
903     {
904         #1 .undefine:,
905         #1 .meta:n =
906         {
907             symbol=#2,
908             llimit=##1
909         },
910     }
911 }

```

12.4 Configuration macros

\IntegralSetup

This macro is documented in section 6.

```

912 \NewDocumentCommand\IntegralSetup{ m }
913 { \keys_set:nn { IntegralSetup } { #1 } }

```

\intexgralsetup

Just like the previous macro, but without forgetting to restrict the usage to the preamble.

This macro is documented in section 1.

```

914 \NewDocumentCommand\intexgralsetup{ m }
915   { \keys_set:nn { intexgral } { #1 } }
916 \@onlypreamble\intexgralsetup

```

\invertdiff

Just a trivial definition that simply inverts the boolean seen at the beginning of the implementation.

```

917 \NewDocumentCommand\invertdiff{ }
918   { \bool_set_inverse:N \l__intexgral_invert_differential_bool }

```

\integral

The final macro is only a few lines long. If an optional argument is provided, it performs the necessary checks to determine the nature of that argument (whether it follows special syntax or not) and assigns the keys accordingly. It then assigns the main argument to the corresponding token before calling the central macro that constructs the entire expression. The entire process is executed within a group so that all modifications remain local.

This macro is documented in section 2.

```

919 \NewDocumentCommand\integral{ 0{} m }
920   {
921     \group_begin:
922     \tl_if_empty:nF { #1 }
923       {
924         \__intexgral_extract_first_key_name:n { #1 }
925         \__intexgral_parse_integral_keys:n { #1 }
926       }
927     \tl_set:Nn \l__intexgral_integrand_tl { #2 }
928     \__intexgral_print_integral:
929     \group_end:
930   }

```

13 Package configuration

The last part of the package is dedicated to defining the default keywords for symbols, limits and variables, as well as the default parameters for the typesetting of integrals.

13.1 Symbol

```

931 \NewSymbolKeyword{single}      {\int}
932 \NewSymbolKeyword{double}     {\iint}
933 \NewSymbolKeyword{triple}     {\iiint}
934 \NewSymbolKeyword{quadruple}  {\iiiiint}
935 \NewSymbolKeyword{contour}    {\oint}
936 \NewSymbolKeyword{surface}    {\oiint}
937 \NewSymbolKeyword{volume}     {\oiint}

```

13.2 Limits

```

938 \NewLimitsKeyword{ab}         {a, b}
939 \NewLimitsKeyword{real}       {-\infty, +\infty}
940 \NewLimitsKeyword{positive}   {0, +\infty}
941 \NewLimitsKeyword{negative}   {-\infty, 0}
942 \NewLimitsKeyword{unit}       {0, 1}
943 \NewLimitsKeyword{circle}     {0, 2\pi}
944 \NewLimitsKeyword{scircle}    {0, \pi}

```

```

945 \NewLimitsKeyword{qcircle}    {0, \pi/2}
946 \NewLimitsKeyword{height}    {0, H}
947 \NewLimitsKeyword{radius}    {0, R}
948 \NewLimitsKeyword{length}    {0, L}
949 \NewLimitsKeyword{time}       {0, T}

```

13.3 Variables

```

950 \NewVariableKeyword{cartesian} {x, y, z}
951 \NewVariableKeyword{planar}    {x, y}
952 \NewVariableKeyword{polar}     {r, \theta} [r]
953 \NewVariableKeyword{cylindrical} {r, \theta, z} [r]
954 \NewVariableKeyword{cylindrical*} {\rho, \theta, z} [\rho]
955 \NewVariableKeyword{spherical} {r, \theta, \phi} [r^2, \sin\theta]

```

13.4 Default parameters

```

956 \IntegralSetup{
957   diff-symb   = \l__intexgral_default_differential_symbol_tl,
958   defaultvar  = {x},
959   defaultvar* = {r},
960   varsep      = \thinmuskip,
961   diffsep     = \thinmuskip,
962   innersymbsep = {-5mu},
963   postsymbsep  = {-1mu},
964   vectorstyle = \vec,
965   domainstyle = \mathbb,
966   novar       = false
967 }
968 \</package>

```

Index

Symbols

_ 93, 97, 101, 105, 109, 113, 118

B

bool commands:

\bool_case:nTF 212
\bool_if:N
.. 216, 250, 267, 283, 312, 314,
340, 421, 438, 446, 458, 461,
463, 471, 482, 488, 491, 496,
509, 513, 529, 534, 547, 550,
556, 567, 574, 582, 784
\bool_if:nTF 409
\bool_lazy_and:nnTF 203
\bool_new:N . 28, 29, 30, 164, 165,
166, 167, 168
\bool_set_false:N . . . 442, 607
\bool_set_inverse:N 918
\bool_set_true:N . 441, 612, 617,
726, 728, 750, 752

C

\c 440

\cdot 462, 489

clist commands:

\clist_if_empty:N 331
\clist_item:Nn 335
\clist_new:N 172

cs commands:

\cs_generate_variant:Nn . 143,
144, 145, 146, 147, 148, 149, 150
\cs_if_exist:N 27, 648
\cs_new:Nn 83
\cs_new:Npn . 183, 184, 209, 210,
346
\cs_new_eq:NN 182
\cs_new_protected:Nn 232, 293,
310, 419, 578
\cs_new_protected:Npn
.. 180, 244, 281, 347, 361, 444,
469, 494, 519, 532, 554, 780, 811
\cs_set_protected:Npn 448, 473,
498, 536, 558

D

\DeclareLimitsKeyword 806

\DeclareSymbolKeyword 900

\DeclareVariableKeyword 843

\def 6, 7, 8, 9

\differentials . . 448, 473, 498, 536,
558

E

exp commands:

\exp_args:Ne 204, 205, 688, 709
\exp_args:NNne 371
\exp_last_unbraced:Ne . . . 218
\exp_last_unbraced:N . . . 355
\exp_not:n . . 285, 286, 289, 290,
330, 341, 693, 714

G

group commands:

\group_begin: 921
\group_end: 929

H

hook commands:

\hook_gput_code:nnn 26

I

\iiiint 934

\iiint 933

\iint 932

\infty 272, 276, 939, 940, 941

\int 127, 153, 652, 664, 931

int commands:

\int_compare:nNnTF . . 380, 387,
658, 665
\int_compare_p:nNn 411
\int_gincr:N 580
\int_gzero_new:N 82
\int_set:Nn 295
\int_step_inline:nn . 301, 504,
521, 542, 564, 661
\int_use:N 86
\l_tmpa_int 295, 301

\integral 919

\IntegralSetup 912, 956

intexgral internal commands:

\l__intexgral_custom_variables_-
bool 165, 312,
726
\l__intexgral_deactivate_-
variables_bool 29, 438, 728,
778
\l__intexgral_default_differential_-
symbol_tl . 31, 55, 60, 69, 74,
957
\l__intexgral_default_variables_-
seq 170, 323, 374, 384,
763

<code>\l__intexgral_default_vectorial_-</code> <code>variables_seq</code> ... 171, 318, 769	<code>\l__intexgral_integral_-</code> <code>symbol_tl</code> 152, 153, 186, 649, 652, 660, 663, 667
<code>\g__intexgral_differential_-</code> <code>group_keyword_prop</code> 174, 731, 813, 824, 830, 840, 845	<code>\l__intexgral_integrand_seq</code> . . 194, 424, 430, 457, 490, 508, 528, 546, 573
<code>\l__intexgral_differential_-</code> <code>order_clist</code> .. 172, 331, 336, 754	<code>\l__intexgral_integrand_tl</code> 151, 426, 431, 440, 927
<code>\l__intexgral_differential_-</code> <code>sep_muskip</code> 178, 451, 466, 476, 485, 501, 516, 774	<code>\l__intexgral_invert_differential_-</code> <code>bool</code> 30, 39, 582, 918
<code>\l__intexgral_differential_-</code> <code>seq</code> 196, 328, 450, 465, 475, 484, 500, 515, 525, 538, 551, 560, 570	<code>__intexgral_invert_limits:w</code> 183, 218, 785
<code>\l__intexgral_differential_-</code> <code>symbol_tl</code> ... 156, 330, 753, 759	<code>\l__intexgral_invert_limits_-</code> <code>bool</code> 28, 35, 216, 250, 267, 283, 784
<code>\l__intexgral_display_mode_-</code> <code>str</code> .. 179, 584, 593, 608, 613, 618	<code>\l__intexgral_jacobian_bool</code> . . 168, 458, 491, 509, 529, 547, 574, 750
<code>\l__intexgral_domain_char_tl</code> .. 159, 687, 694, 708, 715	<code>\l__intexgral_jacobian_seq</code> 195, 459, 492, 510, 530, 548, 575, 740
<code>\l__intexgral_domain_dimension_-</code> <code>tl</code> 160, 689, 695, 710, 716	<code>\l__intexgral_key_name_tl</code> 163, 353, 359, 363
<code>\l__intexgral_domain_seq</code> 169, 682, 683, 703, 704	<code>\c__intexgral_left_bracket_tl</code> 161, 273, 276
<code>\l__intexgral_domain_style_tl</code> 158, 693, 714, 776	<code>\g__intexgral_limits_keyword_-</code> <code>prop</code> 173, 214, 219, 222, 782, 791, 797, 803, 808, 851, 868, 886
<code>__intexgral_extract_first_-</code> <code>key_name:n</code> 347, 924	<code>\l__intexgral_limits_seq</code> 198, 234, 246, 630, 637
<code>__intexgral_general_integral_-</code> <code>symbol:</code> 184, 307, 436	<code>\l__intexgral_limits_style_tl</code> .. 32, 45, 47, 187, 624, 626
<code>__intexgral_generate_integral_-</code> <code>sequence:</code> 293, 632, 639	<code>\l__intexgral_lower_limit_tl</code> 154, 189, 252, 260, 274, 285, 289, 642, 685, 686, 691, 706, 707, 712, 722
<code>__intexgral_has_pm:n</code> ... 201	<code>\l__intexgral_lower_limits_-</code> <code>seq</code> . 200, 238, 269, 297, 299, 304
<code>__intexgral_has_pm_p:n</code> . 224	<code>\l__intexgral_manual_differential_-</code> <code>bool</code> 164, 441, 442, 446, 463, 471, 482, 496, 513, 534, 550, 556, 567
<code>\g__intexgral_integral_-</code> <code>number_int</code> 82, 86, 580	<code>__intexgral_mathchoice:nnnn</code> 182, 669
<code>__intexgral_integral_preconfiguration\</code> 419, 581	<code>__intexgral_mkern:N</code> . 180, 451, 455, 456, 460, 466, 476, 480, 481, 485, 487, 501, 507, 512, 516, 524, 527, 545, 549, 569, 572
<code>\l__intexgral_integral_-</code> <code>symbol_seq</code> 193, 306, 433, 435, 454, 479, 504, 506, 521, 523, 542, 544, 564, 566	

`\__intexgral_new_limits_-`
`group:nn` . 780, 793, 798, 804, 809
`\__intexgral_new_variables_-`
`group:nnn` 811, 826, 834, 841, 847
`\__intexgral_parse_integral_-`
`keys:n` 361, 925
`\__intexgral_parse_limits:n` .
`.....` 210, 237, 249
`\__intexgral_parse_variables:`
`.....` 310, 439
`\__intexgral_print_default_-`
`integral:` 444, 595, 599
`\__intexgral_print_default_-`
`integral_inv:` .. 469, 586, 590
`\__intexgral_print_integral:`
`.....` 578, 928
`\__intexgral_print_nested_-`
`integral:` 494, 596
`\__intexgral_print_nested_-`
`integral_inv:` 519, 587
`\__intexgral_print_product_-`
`integral:` 532, 597
`\__intexgral_print_product_-`
`integral_inv:` 554, 588
`\__intexgral_retrieve_key_-`
`name:w` 346, 356
`\c__intexgral_right_bracket_-`
`tl` 162, 272, 277
`\l__intexgral_separate_-`
`integral_bool` 166, 421, 607, 612, 617
`\__intexgral_set_limits:nn` ..
`.....` 281, 303
`\__intexgral_set_limits_-`
`regular:` 232, 631
`\__intexgral_set_limits_-`
`starred:` 244, 638
`\l__intexgral_symbol_inner_-`
`sep_muskip` .. 175, 455, 480, 777
`\l__intexgral_symbol_post_-`
`sep_muskip` 176, 456, 481, 507, 524, 545, 569, 772
`\__intexgral_trim_pm:w` .. 209, 226, 227
`\l__intexgral_upper_limit_tl`
`..` 155, 191, 286, 290, 644
`\l__intexgral_upper_limits_-`
`seq` 199, 240, 268, 298, 305
`\l__intexgral_variable_sep_-`
`muskip` 177, 460, 487, 512, 527, 549, 572, 773
`\l__intexgral_variables_seq` .
`..` 197, 317, 322, 326, 734, 745
`\l__intexgral_vectorial_-`
`differential_bool` 167, 314, 340, 461, 488, 752
`\l__intexgral_vectorial_-`
`differential_style_tl` 157, 341, 775
`\__intexgral_warning_msg_-`
`header:` .. 83, 118, 125, 132, 139
`\intexgralsetup` 914, 916
`\invertdiff` 917
`iow commands:`
`\iow_newline:` 19, 20, 89

K
`keys commands:`
`\keys_define:nn` .. 33, 602, 757, 856, 872, 890, 902
`\keys_if_exist:nnTF` . 145, 363, 853, 870, 888
`\keys_set:nn` 364, 392, 913, 915

M
`\mathbb` 27, 965
`\mathchoice` 182
`\mathnormal` 56, 75
`\mathrm` 61, 70
`msg commands:`
`\msg_critical:nn` 25
`\msg_error:nnn` .. 792, 799, 825, 836, 854, 882
`\msg_line_context:` 87
`\msg_new:nnn` 16, 115, 123, 130, 137
`\msg_new:nnnn` 91, 95, 99, 103, 107, 111

\msg_see_documentation_text:n
 21
 \msg_warning:nn 659
 \msg_warning:nnn . 651, 743, 852,
 869, 887

muskip commands:

\muskip_new:N 175, 176, 177, 178

N

\NeedsTeXFormat 3
 \NewDocumentCommand . 789, 795, 801,
 806, 822, 828, 838, 843, 849,
 866, 884, 900, 912, 914, 917, 919
 \NewLimitsKeyword 113, 118, 789, 938,
 939, 940, 941, 942, 943, 944,
 945, 946, 947, 948, 949
 \NewSymbolKeyword 97, 849, 931, 932,
 933, 934, 935, 936, 937
 \NewVariableKeyword . 822, 950, 951,
 952, 953, 954, 955
 \NewVarKeyword 105

O

\oiint 937
 \oint 936
 \oint 935

P

\partial 722
 \phi 955
 \pi 943, 944, 945

prg commands:

\prg_new_conditional:Npnn 201
 \prg_return_false: 207
 \prg_return_true: 206

\ProcessKeyOptions 81

prop commands:

\prop_get:NnNTF 730
 \prop_gput:Nnn ... 148, 782, 813
 \prop_if_in:NnTF 791, 803, 824,
 840, 851, 868, 886
 \prop_if_in_p:Nn 214
 \prop_item:Nn 219, 222
 \prop_new:N 173, 174
 \prop_pop:NnNTF 797, 830
 \prop_remove:Nn 808, 845

\ProvideLimitsKeyword 801
 \ProvidesExplPackage 11
 \ProvideSymbolKeyword 884
 \ProvideVariableKeyword 838

Q

quark commands:

\q_stop 183, 209, 220, 226, 227,
 346, 356, 785

R

regex commands:

\regex_split:nnN 366
 \RenewLimitsKeyword 109, 795
 \RenewSymbolKeyword 93, 866
 \RenewVariableKeyword 828
 \RenewVarKeyword 101
 \RequirePackage 4, 27
 \rho 954

S

scan commands:

\scan_stop: . 181, 670, 671, 672,
 673

seq commands:

\seq_count:N 298, 299, 380, 387,
 411, 504, 521, 542, 564
 \seq_gclear:N 833, 846
 \seq_gpop_left:NN ... 538, 560
 \seq_gset_split:Nnn . 149, 819
 \seq_if_empty:NTF ... 297, 433
 \seq_if_exist:NTF ... 737, 817
 \seq_item:Nn 239,
 241, 254, 256, 262, 264, 271,
 275, 304, 305, 350, 368, 391,
 394, 395, 398, 413, 506, 508,
 510, 523, 525, 544, 546, 548,
 551, 566, 570, 573, 575
 \seq_map_indexed_inline:Nn ..
 246, 326
 \seq_map_inline:Nn . 234, 683,
 704
 \seq_new:N 169, 170, 171, 193, 194,
 195, 196, 197, 198, 199, 200, 818
 \seq_put_right:Nn 238,
 240, 268, 269, 306, 328, 382,
 388, 429, 435
 \seq_set_eq:NN ... 316, 321, 739
 \seq_set_item:Nnn 371
 \seq_set_split:Nnn .. 150, 236,
 248, 349, 389, 423, 630, 637,
 682, 703, 733, 745, 762, 768
 \seq_use:Nn 146, 373,
 384, 450, 454, 457, 459, 465,
 475, 479, 484, 490, 492, 500,
 515, 528, 530

`\l_tmpa_seq` . 236, 239, 241, 248,
 254, 256, 262, 264, 271, 275,
 349, 350, 366, 368, 371, 380,
 383, 387, 388, 391, 394, 398
`\l_tmpb_seq` . . 389, 395, 411, 413
`\sin` 955
 str commands:
`\str_case:nnTF` . . 398, 584, 593
`\str_case_e:nnTF` 271, 275
`\str_if_eq:nnTF` . . 143, 144, 367,
 727
`\str_if_eq_p:nn` . 147, 204, 205,
 413
`\str_item:nn` 204, 205
`\str_new:N` 179
`\str_set:Nn` 608, 613, 618
`\str_uppercase:n` 688, 709

T
 T_EX and L^AT_EX 2_ε commands:
`\@ifpackagelater` 24
`\@onlypreamble` 916
`\intexgral@date` 8, 13
`\intexgral@description` . . 9, 15
`\intexgral@module` 6, 12
`\intexgral@version` 7, 14
 tex commands:
`\tex_left:D` 272, 273
`\tex_limits:D` 45, 624
`\tex_mathpunct:D` 255, 263
`\tex_mkern:D` 181, 670, 671, 672,
 673
`\tex_nolimits:D` 47, 626
`\tex_right:D` 276, 277
`\tex_unkern:D` 462, 489
`\theta` 952, 953, 954, 955
`\thinmuskip` 960, 961
`\times` 686, 707

tl commands:
`\c_space_tl` 86
`\tl_clear:N` 660
`\tl_const:Nn` 161, 162
`\tl_head:n` 688, 709
`\tl_if_blank:nTF` 815
`\tl_if_empty:NnTF` 685, 706
`\tl_if_empty:nTF` 922
`\tl_if_in:NnTF` 351
`\tl_if_regex_match:nnTF` . 440
`\tl_new:N` 31, 32, 151, 152, 154, 155,
 156, 157, 158, 159, 160, 163
`\tl_put_right:Nn` 663, 667, 686,
 691, 707, 712
`\tl_set:Nn` 55,
 60, 69, 74, 252, 260, 285, 286,
 289, 290, 350, 353, 681, 687,
 689, 702, 708, 710, 722, 927
`\tl_set_eq:NN` . 45, 47, 153, 359,
 624, 626, 649, 652
`\tl_tail:n` 690, 711
`\tl_trim_spaces:n` . 92, 96, 100,
 104, 108, 112, 126
`\tl_use:N` 274, 539, 561
`\l_tmpa_tl` 350, 351, 356, 359, 538,
 539, 560, 561, 681, 682, 702,
 703, 731, 736, 797, 831
 token commands:
`\c_math_subscript_token` . 188,
 716
`\c_math_superscript_token` 190,
 333, 695
`\token_to_str:N` 93, 97, 101, 105,
 109, 118, 127

V
`\vec` 964